

TRƯỜNG ĐẠI HỌC MỞ HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
MÔN NHẬP MÔN CÔNG NGHỆ PHẦN MỀM
ĐỀ TÀI: XÂY DỰNG PHẦN MỀM QUẢN LÝ BÁN HÀNG CỬA HÀNG
ĐỒ ĂN NHANH FAF

Giảng viên hướng dẫn: Trương Hữu Thành

Sinh viên thực hiện:

NHÓM 5:

Vũ Việt Anh	- 23A1001D0028
Nguyễn Viết Chung	- 23A1001D0054
Trần Khánh Duy	- 23A1001D0109
Nguyễn Hoàng Hải Đăng	- 23A1001D0061
Phan Quang Hiếu	- 23A1001D0144
Phạm Bá Huy	- 23A1001D0185
Lê Thiên Phú	- 23A1001D0288
Nguyễn Hữu Tín	- 23A1001D0359

Hà Nội, 2026

LỜI NÓI ĐẦU

Trong bối cảnh công nghệ thông tin ngày càng phát triển mạnh mẽ và được ứng dụng sâu rộng vào mọi lĩnh vực của đời sống, việc tin học hóa các hoạt động quản lý đã trở thành nhu cầu tất yếu đối với các doanh nghiệp, đặc biệt là trong lĩnh vực kinh doanh dịch vụ ăn uống. Các cửa hàng đồ ăn nhanh với đặc trưng là số lượng đơn hàng lớn, tốc độ phục vụ cao và danh mục sản phẩm đa dạng đòi hỏi một hệ thống quản lý bán hàng hiệu quả, chính xác và linh hoạt nhằm nâng cao chất lượng dịch vụ cũng như năng lực cạnh tranh trên thị trường.

Xuất phát từ thực tế đó, nhóm chúng em lựa chọn đề tài “Xây dựng phần mềm quản lý bán hàng cho cửa hàng đồ ăn nhanh FAF” cho bài tập lớn môn *Nhập môn Công nghệ Phần mềm*. Đề tài tập trung nghiên cứu bài toán quản lý các nghiệp vụ cơ bản của một cửa hàng đồ ăn nhanh như: quản lý sản phẩm, quản lý đơn hàng, thanh toán, thống kê – báo cáo doanh thu, đồng thời hỗ trợ quy trình bán hàng một cách nhanh chóng và thuận tiện.

Thông qua việc phân tích yêu cầu, mô hình hóa hệ thống và đề xuất giải pháp phần mềm phù hợp, đề tài không chỉ giúp chúng em vận dụng những kiến thức đã học về quy trình phát triển phần mềm, phân tích nghiệp vụ và thiết kế hệ thống, mà còn góp phần nâng cao khả năng tư duy logic, làm việc nhóm và tiếp cận các bài toán thực tế trong lĩnh vực công nghệ thông tin.

Mặc dù đã có nhiều cố gắng trong quá trình thực hiện, bài tập lớn khó tránh khỏi những hạn chế và thiếu sót do thời gian và kinh nghiệm còn hạn chế. Nhóm chúng em rất mong nhận được sự góp ý và đánh giá của giảng viên để đề tài được hoàn thiện hơn.

MỤC LỤC

LỜI NÓI ĐẦU.....	1
MỤC LỤC.....	2
CHƯƠNG 1. GIỚI THIỆU.....	5
1.1. Tổng quan về đề tài.....	5
1.2. Giới thiệu bài toán.....	5
1.3. Mục tiêu của đề tài.....	6
1.4. Các đề tài/hệ thống liên quan.....	8
1.5. Phương pháp phát triển.....	9
1.6. Vai trò của thành viên dự án.....	11
1.7. Phân công nhiệm vụ.....	11
1.8. Lập kế hoạch triển khai chi tiết.....	14
CHƯƠNG 2. SẢN PHẨM CHUYỂN GIAO GIỮA CÁC PHA VÀ TIÊU	
CHUẨN ĐÁNH GIÁ.....	20
2.1. Danh mục tài liệu, sản phẩm.....	20
2.2. Quy tắc đặt tên (naming convention).....	23
2.3. Xác định Mục tiêu chất lượng của dự án.....	26
2.4. Xác định tiêu chuẩn Kiểm thử được (Testability) cho dự án.....	29
2.4.1. Khái niệm, ý nghĩa của đặc tính Kiểm thử được.....	29
2.4.2. Chi tiết hóa các tiêu chí của đặc tính Kiểm thử được.....	30
CHƯƠNG 3. XÁC ĐỊNH YÊU CẦU.....	31
3.1. Vai trò của việc Xác định yêu cầu đối với dự án phần mềm.....	31
3.2. Phương pháp đánh giá chất lượng của yêu cầu.....	31
3.2.1. Tiêu chí đánh giá yêu cầu phần mềm.....	32
3.2.2. Phương pháp kiểm tra và xác nhận yêu cầu.....	35
3.2.3. Công cụ hỗ trợ đánh giá yêu cầu phần mềm.....	41
3.3. Danh sách các yêu cầu của hệ thống.....	47
3.3.1. Yêu cầu chức năng.....	47
3.3.2. Yêu cầu phi chức năng.....	48
3.3.3. Ràng buộc hệ thống.....	49
3.4. Kết quả đánh giá chất lượng các yêu cầu.....	50
3.4.1. Mức độ phù hợp với kết quả khảo sát và định hướng đề tài.....	50
3.4.2. Mức độ bao quát các chức năng chính của hệ thống.....	50

3.4.3. Cấu trúc trình bày yêu cầu rõ ràng và hợp lý.....	51
3.4.4. Chất lượng của nhóm yêu cầu chức năng.....	51
3.4.5. Chất lượng của nhóm yêu cầu phi chức năng.....	52
3.4.6. Mức độ phù hợp với phạm vi hệ thống.....	52
3.4.7. Mức độ rõ ràng và dễ hiểu của yêu cầu.....	53
3.4.8. Mức độ hỗ trợ cho các giai đoạn phát triển tiếp theo.....	53
3.4.9. Những điểm còn hạn chế cần hoàn thiện thêm.....	54
3.4.10. Kết luận đánh giá.....	54
CHƯƠNG 4. KIẾN TRÚC PHẦN MỀM.....	55
4.1. Vai trò của kiến trúc phần mềm với quá trình phát triển dự án phần mềm.....	55
4.2. Một số kiến trúc phần mềm phổ biến.....	56
4.2.1. Monolithic Architecture.....	56
4.2.2. Layered Architecture.....	56
4.2.3. Microservices Architecture.....	57
4.2.4. Three-tier Architecture.....	57
4.3. Kiến trúc tổng thể của hệ thống.....	57
4.3.1. Mô hình Client–Server.....	58
4.3.2. Kiến trúc phân lớp (Layered Architecture).....	58
4.3.3. Lý do lựa chọn kiến trúc này cho hệ thống FAF.....	60
4.3.4. Kết luận.....	60
4.4. Lựa chọn công nghệ phát triển cho hệ thống.....	60
4.4.1. Công nghệ cho lớp giao diện (Presentation Layer).....	61
4.4.2. Công nghệ cho lớp xử lý nghiệp vụ (Business Logic Layer).....	61
4.4.3. Công nghệ cho lớp cơ sở dữ liệu (Database Layer).....	62
4.4.4. Công nghệ giao tiếp giữa các tầng.....	62
4.5. Kiến trúc phần mềm của hệ thống.....	63
4.5.1. Mô hình Client–Server.....	63
4.5.2. Chi tiết các tầng trong kiến trúc phân lớp.....	64
4.5.3. Luồng xử lý dữ liệu (Interaction Flow).....	64
4.5.4. Lợi ích đối với dự án FAF.....	65
4.6. Cơ sở dữ liệu mức vật lý của hệ thống.....	66
4.6.1. Hệ quản trị cơ sở dữ liệu.....	66
4.6.2. Mô hình lưu trữ dữ liệu.....	67

4.6.3. Cấu trúc các bảng chính.....	67
4.6.4. Tối ưu hóa truy vấn.....	73
4.6.5. Sao lưu và phục hồi dữ liệu.....	74
4.7. Một số thiết kế chi tiết theo kiến trúc được chọn.....	75
CHƯƠNG 5. XÂY DỰNG VÀ KIỂM THỬ HỆ THỐNG.....	79
5.1. Kết quả xây dựng module/nhóm chức năng.....	79
5.2. Kiểm thử hệ thống.....	81
5.2.1. Tổng quan về kiểm thử phần mềm.....	81
5.2.2. Mô hình kiểm thử được áp dụng trong đề tài (Agile Testing).....	81
5.2.3. Kế hoạch và thực thi kiểm thử theo quy trình Agile.....	82
5.2.4. Kết quả kiểm thử.....	89
5.2.4.1. Thống kê tổng quát.....	89
5.2.4.2. Phân tích kết quả theo các phân hệ nghiệp vụ.....	89
5.2.4.3. Danh sách các lỗi tiêu biểu và tình trạng khắc phục.....	90
5.2.4.4. Đánh giá chất lượng hệ thống (Final Quality Assessment).....	90
CHƯƠNG 6. KẾT LUẬN.....	93
6.1. Đánh giá chất lượng phần mềm.....	93
6.2. Ưu điểm & Nhược điểm của hệ thống.....	93
6.2.1. Ưu điểm.....	93
6.2.2. Nhược điểm (Hạn chế).....	93
6.3. Hướng phát triển.....	94
TÀI LIỆU THAM KHẢO.....	94
PHỤ LỤC A: QUY TẮC ĐẶT TÊN.....	95
PHỤ LỤC B: CÁC BIỂU MẪU, TÀI LIỆU THU THẬP ĐƯỢC.....	97
PHỤ LỤC C: CÁC BIÊN BẢN TIẾN ĐỘ.....	97
PHỤ LỤC D: CÁC BIÊN BẢN RÀ SOÁT KỸ THUẬT.....	98
PHỤ LỤC E: HỒ SƠ TOÀN BỘ DỰ ÁN.....	100

CHƯƠNG 1. GIỚI THIỆU

1.1. Tổng quan về đề tài

Kinh doanh cửa hàng đồ ăn nhanh là một lĩnh vực có tiềm năng phát triển mạnh mẽ trong thời đại hiện nay, khi nhu cầu tiêu dùng nhanh, tiện lợi ngày càng được ưa chuộng, đặc biệt là tại các khu đô thị và khu vực có mật độ dân cư cao.

Tuy nhiên, việc xây dựng một thương hiệu đồ ăn nhanh bền vững đòi hỏi phải có chiến lược phát triển rõ ràng và lâu dài. Ba yếu tố quan trọng không thể thiếu góp phần tạo nên thành công cho mô hình kinh doanh này bao gồm: Chất lượng món ăn, quy trình phục vụ – quản lý chuyên nghiệp và chiến lược marketing hiệu quả, phù hợp với từng phân khúc khách hàng.

Xuất phát từ thực tế đó, chúng em chọn đề tài “Quản lý bán hàng tại cửa hàng đồ ăn nhanh” với mong muốn xây dựng một giải pháp quản lý phù hợp, khắc phục những khó khăn của phương pháp thủ công hiện tại. Hệ thống sẽ giúp tối ưu hóa quy trình, doanh thu và góp bán hàng, theo dõi tồn kho phần nâng cao hiệu quả quản lý, từ đó hỗ trợ mở rộng quy mô kinh doanh trong tương lai.

1.2. Giới thiệu bài toán

Trong bối cảnh ngành dịch vụ ăn uống (F&B) phát triển mạnh mẽ, mô hình kinh doanh đồ ăn nhanh đang trở thành xu hướng tất yếu, đáp ứng nhu cầu tiện lợi của nhịp sống hiện đại. Việc quản lý quy trình bán hàng và vận hành hiệu quả là yếu tố then chốt quyết định sự thành công và năng lực cạnh tranh của cửa hàng. Một hệ thống quản lý tối ưu không chỉ giúp nâng cao tốc độ phục vụ, đảm bảo sự chính xác trong khâu chế biến mà còn mang lại trải nghiệm dịch vụ tốt nhất cho khách hàng. Tuy nhiên, trên thực tế, nhiều cửa hàng, đặc biệt là các cơ sở kinh doanh quy mô vừa và nhỏ, vẫn chưa chú trọng đầu tư vào các giải pháp công nghệ đồng bộ, dẫn đến quy trình vận hành còn tồn tại nhiều bất cập và thiếu tính chuyên nghiệp.

Hiện nay, đa số các cửa hàng vẫn duy trì phương pháp quản lý truyền thống như ghi chép order bằng giấy, truyền miệng giữa bộ phận phục vụ và nhà bếp, hoặc sử dụng các công cụ tính toán rời rạc như Excel. Cách làm này tuy đơn giản và tiết kiệm chi phí đầu tư ban đầu nhưng bộc lộ nhiều hạn chế nghiêm trọng như thông tin không được đồng bộ tức thời, dễ xảy ra sai sót, nhầm lẫn món ăn và khó khăn trong việc truy xuất lịch sử giao dịch. Khi lượng khách hàng tăng cao vào các khung giờ cao điểm, việc quản lý bằng phương pháp thủ công sẽ trở nên quá tải, gây ách tắc trong quy trình phục vụ và ảnh hưởng trực tiếp đến doanh thu cũng như uy tín của cửa hàng.

Chính vì vậy, việc nghiên cứu và xây dựng một hệ thống phần mềm quản lý bán hàng tại cửa hàng đồ ăn nhanh là vô cùng cần thiết. Một phần mềm chuyên dụng sẽ giúp doanh nghiệp chuẩn hóa quy trình nghiệp vụ, theo dõi trạng thái đơn hàng và nguyên liệu chính xác theo thời gian thực, đồng thời tự động hóa việc tính toán và báo cáo doanh thu. Điều này không chỉ giúp tiết kiệm thời gian, giảm thiểu tối đa các sai sót do con người mà còn nâng cao hiệu suất làm việc tổng thể, hỗ trợ các nhà quản lý nắm bắt tình hình kinh doanh nhanh chóng để đưa ra các quyết định chiến lược chính xác hơn.

1.3. Mục tiêu của đề tài

Mục tiêu của đề tài là xây dựng một hệ thống phần mềm quản lý bán hàng tại cửa hàng đồ ăn nhanh hiện đại, giúp cửa hàng tối ưu hóa quy trình vận hành từ khâu gọi món đến chế biến và thanh toán, đảm bảo sự chính xác và tốc độ phục vụ. Bên cạnh đó, đề tài cũng hướng đến việc thực hiện quy trình phát triển phần mềm chuẩn mực theo yêu cầu môn học. Cụ thể, hệ thống hướng đến các mục tiêu sau:

- **Quản lý bán hàng và thực đơn linh hoạt, chính xác**

- Xây dựng cơ sở dữ liệu lưu trữ thông tin chi tiết về món ăn, combo, giá bán, hình ảnh minh họa và danh mục phân loại (đồ ăn, thức uống, món kèm).
- Hỗ trợ nhân viên thu ngân/phục vụ thực hiện thao tác gọi món (order) nhanh chóng trên giao diện trực quan, giảm thiểu sai sót so với ghi chép thủ công.

- Tự động tính toán tổng tiền, áp dụng các mã giảm giá hoặc chương trình khuyến mãi (nếu có) một cách chính xác.

● **Tự động hóa quy trình kết nối Bếp – Thu ngân**

- Cung cấp cơ chế đồng bộ dữ liệu thời gian thực: ngay khi order được tạo, thông tin món ăn được chuyển trực tiếp xuống màn hình hoặc máy in tại khu vực Bếp.
- Loại bỏ độ trễ và sai sót trong quá trình truyền tin giữa nhân viên phục vụ và đầu bếp.
- Hỗ trợ theo dõi trạng thái chế biến (Đang làm, Đã xong, Đã trả món) để nhân viên phục vụ kịp thời mang ra cho khách.

● **Quản lý kho và định lượng nguyên liệu hiệu quả**

- Hỗ trợ thiết lập công thức định lượng cho từng món ăn (Ví dụ: 1 Burger = 1 vỏ bánh + 1 thịt + 10g sốt).
- Tự động trừ kho nguyên liệu ngay khi món ăn được bán ra, giúp số liệu tồn kho luôn cập nhật sát với thực tế.
- Cảnh báo các nguyên liệu sắp hết hoặc sắp hết hạn sử dụng để có kế hoạch nhập hàng kịp thời.

● **Hỗ trợ báo cáo và ra quyết định kinh doanh**

- Tự động tổng hợp dữ liệu và tạo báo cáo về doanh thu bán hàng theo ca, ngày, tháng hoặc theo từng nhân viên.
- Thống kê các món ăn bán chạy nhất và các món ít người gọi để chủ cửa hàng điều chỉnh thực đơn phù hợp.
- Cung cấp biểu đồ trực quan giúp đánh giá hiệu quả kinh doanh và dòng tiền của cửa hàng.

● **Phân quyền người dùng và bảo mật hệ thống**

- Phân quyền rõ ràng theo vai trò nghiệp vụ: Quản trị viên (Admin), Thu ngân, Nhân viên Bếp, Thủ kho.
- Đảm bảo mỗi nhân viên chỉ thao tác trên các chức năng được cấp phép, ngăn chặn các hành vi gian lận hoặc sửa đổi dữ liệu trái phép.

- Cơ chế xác thực đăng nhập an toàn để bảo vệ dữ liệu kinh doanh .

- **Đáp ứng yêu cầu quy trình Công nghệ phần mềm**

- Thực hiện dự án tuân thủ nghiêm ngặt các bước trong vòng đời phát triển phần mềm (SDLC), từ khảo sát hiện trạng, phân tích yêu cầu đến thiết kế và kiểm thử.
- Xây dựng hệ thống tài liệu kỹ thuật chuẩn (Đặc tả yêu cầu - SRS, Thiết kế hệ thống - SDD) thay vì chỉ tập trung vào viết mã nguồn.
- Áp dụng các tiêu chuẩn kiểm thử phần mềm để đảm bảo hệ thống vận hành ổn định, ít lỗi trước khi đưa vào sử dụng thực tế.

1.4. Các đề tài/hệ thống liên quan

1. Hệ thống của KFC

- Ưu điểm:

Quản lý định lượng : Đây là điểm mạnh nhất. Khi nhân viên bấm bán 1 miếng gà, hệ thống tự động trừ đúng số lượng gà trong kho lạnh. Điều này giúp kiểm soát thất thoát tuyệt đối.

Phân quyền chặt chẽ: Nhân viên chỉ được bán hàng. Các thao tác như hủy hóa đơn, giảm giá cho khách phải có thẻ từ hoặc mã của Quản lý cửa hàng quét vào mới thực hiện được.

Tốc độ in bill: Hệ thống kết nối máy in nhiệt tốc độ cao, bill ra ngay lập tức để chuyển qua bộ phận bếp

-Nhược điểm:

Giao diện phức tạp: Do quá nhiều tính năng quản lý, giao diện thường có nhiều lớp menu con, khiến nhân viên mới mất nhiều thời gian để làm quen.

Cứng nhắc: Khó thay đổi nhanh các chương trình khuyến mãi nếu không có sự can thiệp từ hệ thống tổng

2. Hệ thống của Jollibee

- Ưu điểm:

Giao diện trực quan : Màn hình POS sử dụng các biểu tượng món ăn (hình ảnh gà rán, mì Ý) thay vì chỉ có chữ. Nhân viên nhìn hình bấm chọn rất nhanh, giảm thiểu nhầm lẫn.

Xử lý Combo thông minh: Hệ thống tự động nhảy ra các lựa chọn "Up-size" (ví dụ: "Anh/chị có muốn nâng cấp lên khoai tây lớn không?") giúp nhân viên không quên mời chào khách.

Hoạt động Offline: POS tại cửa hàng thường có chế độ lưu trữ tạm thời nếu mất mạng, đảm bảo việc bán hàng không bị gián đoạn, sau đó sẽ đồng bộ dữ liệu lên cloud sau.

- Nhược điểm:

Quản lý kho đôi khi bị trễ: Do ưu tiên tốc độ bán hàng, đôi khi việc trừ kho nguyên liệu chi tiết (tới từng gram tương cà, tương ớt) không được khớp 100% theo thời gian thực như KFC.

1.5. Phương pháp phát triển

Phương pháp phát triển Agile/Scrum

Agile là một phương pháp phát triển phần mềm linh hoạt, tập trung vào việc thích ứng nhanh với các thay đổi trong quá trình phát triển và nâng cao sự hợp tác giữa các thành viên trong nhóm cũng như với khách hàng. Khác với các mô hình phát triển truyền thống, Agile chia quá trình phát triển thành các giai đoạn nhỏ, cho phép nhóm phát triển liên tục cải tiến và hoàn thiện sản phẩm thông qua các vòng lặp ngắn.

Trong số các phương pháp Agile, Scrum là một framework được sử dụng phổ biến nhằm quản lý và tổ chức công việc của nhóm phát triển. Scrum giúp các nhóm làm việc hiệu quả thông qua việc chia dự án thành các chu kỳ phát triển ngắn gọi là **Sprint**, thường kéo dài từ 2 đến 4 tuần. Sau mỗi Sprint, nhóm phát triển sẽ

tạo ra một phần chức năng hoàn chỉnh của sản phẩm, từ đó có thể nhận phản hồi từ khách hàng và điều chỉnh kịp thời.

Trong Scrum, quá trình phát triển được tổ chức thông qua ba vai trò chính. **Product Owner** là người đại diện cho khách hàng và chịu trách nhiệm xác định yêu cầu của sản phẩm cũng như quản lý danh sách các chức năng cần phát triển (Product Backlog). **Scrum Master** có nhiệm vụ đảm bảo nhóm tuân thủ đúng quy trình Scrum, hỗ trợ nhóm giải quyết các trở ngại và cải thiện hiệu quả làm việc. **Development Team** là nhóm chịu trách nhiệm trực tiếp phát triển sản phẩm, bao gồm các hoạt động như thiết kế, lập trình và kiểm thử.

Ngoài ra, Scrum còn sử dụng một số thành phần quan trọng để quản lý công việc. **Product Backlog** là danh sách toàn bộ các yêu cầu và chức năng của sản phẩm. **Sprint Backlog** là tập hợp các công việc mà nhóm phát triển cam kết thực hiện trong một Sprint. Sau mỗi Sprint, nhóm sẽ tạo ra một **Increment**, tức là phần sản phẩm đã hoàn thiện và có thể sử dụng được.

Quy trình Scrum bao gồm nhiều hoạt động chính nhằm đảm bảo quá trình phát triển diễn ra hiệu quả. Đầu tiên là **Sprint Planning**, nơi nhóm xác định các công việc cần thực hiện trong Sprint. Trong suốt quá trình phát triển, nhóm sẽ tổ chức **Daily Scrum** – cuộc họp ngắn hằng ngày để cập nhật tiến độ và giải quyết các vấn đề phát sinh. Khi Sprint kết thúc, nhóm tiến hành **Sprint Review** để trình bày kết quả đạt được cho các bên liên quan. Cuối cùng, nhóm thực hiện **Sprint Retrospective** nhằm đánh giá lại quá trình làm việc và đề xuất các cải tiến cho Sprint tiếp theo.

Việc áp dụng phương pháp Agile/Scrum mang lại nhiều lợi ích cho quá trình phát triển phần mềm. Phương pháp này giúp nhóm phát triển dễ dàng thích ứng với sự thay đổi của yêu cầu, cải thiện khả năng giao tiếp giữa các thành viên trong nhóm, đồng thời cho phép phát hiện và khắc phục lỗi sớm trong quá trình phát triển. Nhờ đó, chất lượng sản phẩm được nâng cao và thời gian đưa sản phẩm ra thị trường cũng được rút ngắn.

1.6. Vai trò của thành viên dự án

STT	Họ và tên	rAI	DT
01	Vũ Việt Anh		X
02	Nguyễn Viết Chung	X	
03	Trần Khánh Duy		X
04	Nguyễn Hoàng Hải Đăng		X
05	Phan Quang Hiếu		X
06	Phạm Bá Huy		X
07	Lê Thiên Phú		X
08	Nguyễn Hữu Tín	X	

1.7. Phân công nhiệm vụ

- Vũ Việt Anh: vai trò - Dev team

Nhiệm vụ:

- Hỗ trợ Product Owner thu thập thông tin

- Hỗ trợ lập trình Back-end, xây dựng các chức năng xử lý dữ liệu của hệ thống.
- Tham gia tích hợp các module chức năng với cơ sở dữ liệu.
- Hỗ trợ kiểm tra và sửa lỗi trong quá trình phát triển hệ thống.

- Nguyễn Viết Chung: vai trò - Product Owner

Nhiệm vụ:

- Thu thập và phân tích yêu cầu của hệ thống;
- Xây dựng và quản lý Product Backlog;
- Xác định mức độ ưu tiên của các chức năng;
- Làm việc với các bên liên quan để đảm bảo sản phẩm đáp ứng đúng yêu cầu.

- Trần Khánh Duy: vai trò - Dev team

Nhiệm vụ:

- Hỗ trợ lập trình Back-end cho hệ thống.
- Tham gia xây dựng logic xử lý nghiệp vụ của ứng dụng.
- Phối hợp với các thành viên khác để tích hợp các chức năng vào hệ thống.
- Hỗ trợ kiểm tra và khắc phục lỗi trong quá trình phát triển.

- Nguyễn Hoàng Hải Đăng: vai trò - Dev team

Nhiệm vụ:

- Hỗ trợ Product Owner thu thập thông tin
- Hỗ trợ lập trình Back-end và xây dựng các chức năng xử lý dữ liệu.
- Tham gia thiết kế và tối ưu cấu trúc dữ liệu của hệ thống.
- Hỗ trợ kiểm thử và cải thiện hiệu suất hệ thống.

- Phan Quang Hiếu: vai trò - Dev team

Nhiệm vụ:

- Phát triển giao diện ứng dụng Frontend
- Thiết kế bố cục giao diện đảm bảo thân thiện và dễ sử dụng.
- Tích hợp giao diện với các chức năng Back-end.
- Hỗ trợ kiểm tra và cải thiện trải nghiệm người dùng.

- Phạm Bá Huy: vai trò - Dev team

Nhiệm vụ:

- Phát triển giao diện ứng dụng Frontend
- Hỗ trợ thiết kế và hoàn thiện các thành phần giao diện của ứng dụng.
- Phối hợp với các thành viên Back-end để tích hợp dữ liệu lên giao diện.
- Kiểm tra và tối ưu hiển thị giao diện trên hệ thống.

- Lê Thiên Phú: vai trò - Dev team

Nhiệm vụ:

- Phát triển giao diện người dùng (Frontend) cho hệ thống.
- Hỗ trợ thiết kế các thành phần giao diện và bố cục hiển thị của ứng dụng.
- Phối hợp với các thành viên Back-end để tích hợp dữ liệu và chức năng vào giao diện.
- Kiểm tra và tối ưu hiển thị giao diện nhằm đảm bảo tính thân thiện và dễ sử dụng cho người dùng.

- Nguyễn Hữu Tín: vai trò - Scrum Master

Nhiệm vụ:

- Đảm bảo nhóm tuân thủ đúng quy trình Scrum;
- Tổ chức các cuộc họp Scrum (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective);
- Hỗ trợ nhóm giải quyết các trở ngại trong quá trình phát triển.

1.8. Lập kế hoạch triển khai chi tiết

Product Backlog

ID	Nhóm (Epic)	User Story (Câu chuyện người dùng)	Mức ưu tiên	Sprint
PB01	Đặt món	Là khách hàng, tôi muốn xem menu điện tử trên tablet để tự do chọn món.	Cao nhất	Sprint 1
PB02	Đặt món	Là khách hàng, tôi muốn đặt món trực tiếp trên tablet để không phải gọi bồi bàn.	Cao nhất	Sprint 1
PB03	Đặt món	Là hệ thống, tôi muốn tự động lập đơn đặt món khi khách chọn để đẩy dữ liệu đi xử lý.	Cao nhất	Sprint 1
PB04	Đặt món	Là quản lý, tôi muốn hệ thống gửi đơn đến bếp và thu ngân để các bộ phận phối hợp ngay.	Cao nhất	Sprint 1

PB05	Bếp & Phục vụ	Là đầu bếp, tôi muốn thấy danh sách đơn đặt món để biết bàn nào cần làm món gì.	Cao nhất	Sprint 2
PB07	Bếp & Phục vụ	Là đầu bếp, tôi muốn cập nhật trạng thái (đang làm/xong) để bồi bàn nắm tiến độ.	Cao	Sprint 2
PB08	Bếp & Phục vụ	Là bồi bàn, tôi muốn nhận thông báo khi món xong để mang ra bàn ngay cho khách.	Cao	Sprint 2
PB10	Đặt món	Là khách hàng, tôi muốn đặt thêm món trong lúc ăn để dùng thêm đồ mà không lập đơn mới.	Trung bình	Sprint 2
PB11	Bếp & Phục vụ	Là bồi bàn, tôi muốn xác nhận đã phục vụ món trên hệ thống để tránh mang nhầm/sót.	Trung bình	Sprint 2
PB18	Bếp & Phục vụ	Là khách hàng, tôi muốn đánh giá món ăn trực tiếp trên tablet để nhà hàng ghi nhận phản hồi.	Thấp	Sprint 2

PB06	Thanh toán	Là thu ngân, tôi muốn hệ thống tổng hợp đơn của bàn để lập hóa đơn chính xác.	Cao	Sprint 3
PB09	Thanh toán	Là thu ngân, tôi muốn lập hóa đơn và có nhiều phương thức thanh toán để khách dễ trả tiền.	Cao	Sprint 3
PB12	Thanh toán	Là khách hàng, tôi muốn bấm nút yêu cầu thanh toán trên tablet để thu ngân mang bill tới.	Trung bình	Sprint 3
PB13	Thanh toán	Là thu ngân, tôi muốn có quyền chỉnh sửa hóa đơn để xử lý khi khách trả món hoặc lỗi.	Trung bình	Sprint 3
PB14	Kho bãi	Là thủ kho, tôi muốn quản lý nhập kho từ nhà cung cấp để theo dõi hàng đầu vào.	Trung bình	Sprint 4
PB15	Kho bãi	Là thủ kho, tôi muốn quản lý xuất kho cho bếp và lưu lượng đã dùng để tính chi phí.	Trung bình	Sprint 4

PB16	Kho bãi	Là quản lý, tôi muốn lưu thông tin nhà cung cấp và lịch sử nhập để đối soát.	Trung bình	Sprint 4
PB17	Kho bãi	Là đầu bếp, tôi muốn lập phiếu yêu cầu nguyên liệu để thủ kho xuất hàng.	Thấp	Sprint 4
PB19	Kiểm kê	Là thủ kho, tôi muốn kiểm kê nguyên liệu thực tế định kỳ để đối chiếu với phần mềm.	Thấp	Sprint 5
PB20	Kiểm kê	Là quản lý, tôi muốn xem báo cáo nguyên liệu nhập/xuất/tồn để nắm bắt tình hình thất thoát.	Thấp	Sprint 5
PB21	Kiểm kê	Là thủ kho, tôi muốn hệ thống tự động tạo phiếu yêu cầu bổ sung khi tồn kho thấp để không thiếu hàng.	Thấp	Sprint 5

=> Các sprint chính:

Sprint 1: Trải nghiệm Khách hàng & Gọi món (Customer Core)

Mục tiêu: Khách cầm máy tính bảng là phải gọi được món.

Menu điện tử: Hiện thị danh sách món, giá cả trên tablet.

Đặt món: Khách chọn món, đặt trực tiếp và hệ thống tự động lập đơn.

Giao dịch: Đơn hàng được đẩy xuống bộ phận giao dịch (thu ngân) để quản lý bàn.

Mở rộng: Cho phép khách đặt thêm món trong lúc đang ngồi ăn.

Sprint 2: Vận hành Bếp & Phục vụ (Kitchen & Service)

Mục tiêu: Kết nối luồng dữ liệu từ Tablet xuống Bếp và ra bàn.

Màn hình Bếp: Hiển thị danh sách đơn cần làm cho bộ phận chế biến.

Trạng thái: Bếp cập nhật "Đang chế biến" hoặc "Hoàn thành".

Thông báo: Hệ thống báo cho bồi bàn khi món xong; bồi bàn xác nhận đã phục vụ khách.

Phản hồi: Lưu lại đánh giá của khách về món ăn ngay sau khi phục vụ.

Sprint 3: Thanh toán & Hóa đơn (Billing & Payment)

Mục tiêu: Thu tiền chính xác và xử lý sai sót.

Hóa đơn: Tổng hợp tất cả các đơn nhỏ của một bàn thành một hóa đơn tổng.

Nghiệp vụ: Lập hóa đơn bán, cho phép sửa hóa đơn nếu nhập nhầm hoặc khách đổi ý.

Thanh toán: Tích hợp đa kênh (Tiền mặt, thẻ, ví điện tử, QR Code).

Kết thúc: Giải phóng bàn trên hệ thống sau khi thanh toán xong.

Sprint 4: Quản lý Nhập kho & Nhà cung cấp (Supply Chain)

Mục tiêu: Quản lý "đầu vào" và các bên cung ứng.

Nhà cung cấp: Lưu danh sách NCC và lịch sử các lần nhập hàng.

Nhập kho: Thực hiện quy trình nhập nguyên liệu vào kho.

Yêu cầu: Cho phép các bộ phận (Bếp) lập phiếu yêu cầu nguyên liệu khi cần.

Xuất kho: Quản lý xuất nguyên liệu từ kho cho bộ phận chế biến và lưu vết sử dụng.

Sprint 5: Kiểm kê & Quản trị kho (Inventory Audit)

Mục tiêu: Chống thất thoát và tự động hóa cảnh báo.

Kiểm kê: Chức năng kiểm kê thực tế định kỳ để đối soát với hệ thống.

Báo cáo: Xuất báo cáo tổng hợp nguyên liệu nhập và đã sử dụng (hao hụt).

Tự động hóa: Hệ thống tự tạo phiếu yêu cầu bổ sung khi tồn kho chạm ngưỡng thấp.

Tổng kết: Fix bug toàn hệ thống và tối ưu hóa luồng dữ liệu liên thông.

CHƯƠNG 2. SẢN PHẨM CHUYỂN GIAO GIỮA CÁC PHA VÀ TIÊU CHUẨN ĐÁNH GIÁ

2.1. Danh mục tài liệu, sản phẩm

STT	Loại tài liệu/Sản phẩm	Biểu mẫu	Tiêu chuẩn đánh giá	Pha Requirement	Pha Analysis	Pha Design	Pha Implementation	Pha Testing
1	Kết quả khảo sát thực tế về quy trình nghiệp vụ	Kết quả khảo sát	Đáp ứng được các yêu cầu từ khách hàng	Out	In			
2	Xác định cơ cấu tổ chức		Người dùng và hệ thống	Out	In			
3	Xác định các bên liên quan và yêu cầu của họ		Stakeholders' requests	Out	In			
4	Xác định các yêu cầu chức năng	Yêu cầu chức năng và phi	Yêu cầu chức năng, phi chức năng phải đầy đủ,	Out	In			In

	và phi chức năng	<u>chức năng</u>	rõ ràng, sát với thực tế					
5	Tiến trình nghiệp vụ: sơ đồ tiến trình nghiệp vụ của từng nghiệp vụ		Đặc tả chi tiết từng bước trong quy trình nghiệp vụ	Out	In			
6	Sơ đồ phân rã chức năng	<u>Sơ đồ phân rã chức năng (BFD)</u>	Mô hình hóa được cấu trúc phần mềm		Out	In	In	
7	Đặc tả chức năng	<u>Cách viết đặc tả yêu cầu phần mềm</u>	Đặc tả yêu cầu chức năng phải chỉ rõ được phạm vi của sản phẩm, các chức năng cần có, đối tượng người sử dụng và các ràng buộc khi vận hành sản phẩm		Out		In	In

8	Phân tích hệ thống về dữ liệu				Out		In	
9	Kiến trúc vật lý	Physical Architecture	Các yếu tố vật lý cụ thể phải được xác định có thể hỗ trợ các tính năng chức năng, hành vi và thời gian cũng như các thuộc tính mong đợi của hệ thống được suy ra từ các yêu cầu hệ thống phi chức năng.		Out		In	
10	Kiến trúc logic				Out	In	In	
11	Mô hình CSDL	Chuẩn hóa CSDL	Cơ sở dữ liệu phải đạt chuẩn 1NF, 2NF, 3NF		Out		Out	In
12	Giao diện hệ thống	User-friendly Interface	Bản thiết kế giao diện hệ thống phải thân thiện với người dùng và đầy đủ chức năng			Out	In	In

13	Thiết kế kiểm soát	Control Design Effectiveness	Phải kiểm soát được thông tin nhập xuất, khả năng gián đoạn chương trình và khả năng bị xâm hại bởi con người			Out	In	
14	Thiết kế xử lý chức năng		Đặc tả chi tiết cho từng unit của mỗi chức năng của hệ thống: input, output, process		Out		In	
15	Công cụ, ngôn ngữ, môi trường lập trình						Out	
16	Source code của từng chức năng hệ thống						Out	In
17	Test case							Out
18	Test report							Out

2.2. Quy tắc đặt tên (naming convention)

Tên file tài liệu cần được đặt theo một quy tắc thống nhất nhằm đảm bảo dễ quản lý, truy xuất và phân loại.

Cấu trúc đặt tên file tài liệu của hệ thống như sau:

2.2.1 Tên file tài liệu:

- Cấu trúc: [Loại tài liệu][Tên dự án][Mô tả ngắn]_[Ngày tạo].[Định dạng]

- Trong đó:

[Loại tài liệu]: Viết tắt loại tài liệu

[Tên dự án]: Tên dự án hoặc mã dự án viết ngắn gọn.

[Mô tả ngắn]: Mô tả nội dung tài liệu để dễ phân biệt.

[Ngày tạo]: Định dạng YYYYMMDD (Năm-Tháng-Ngày) để dễ sắp xếp theo thời gian.

[Định dạng]: Phần mở rộng của file, ví dụ: .docx, .xlsx, .pdf.

- Ví dụ: SRS_ECOM_UserModule_20260317.pdf

(SRS=Software Requirements Specification/ ECOM=Ecommerce Project)

2.2.2. Tên biến trong mã nguồn:

Quy tắc: Để đảm bảo mã nguồn dễ đọc, dễ bảo trì và tuân theo các chuẩn lập trình, nhóm sẽ tuân theo quy tắc đặt tên biến sau:

1. Sử dụng tiếng Anh: Tránh đặt tên biến bằng tiếng Việt có dấu hoặc không dấu.

2. Sử dụng kiểu CamelCase hoặc snake_case tùy theo loại biến:

- Biến cục bộ, tham số hàm: dùng camelCase, bắt đầu bằng chữ thường.

- Biến hằng số: dùng UPPER_CASE_SNAKE_CASE, viết hoa toàn bộ và phân cách bằng dấu gạch dưới _.

- Tên lớp, đối tượng: dùng PascalCase, mỗi chữ cái đầu của từ viết hoa.

3. Tên biến phải có ý nghĩa, tránh viết tắt khó hiểu:

- customerName, orderTotal, isValidUser
- cusNm, ordTtl, ivU

4. Tránh sử dụng các tên biến chung chung hoặc khó hiểu:

- productList (danh sách sản phẩm), userAge (tuổi của người dùng)
- data, temp, value

5. Tuân theo quy tắc đặt tên theo ngữ cảnh sử dụng:

2.2.3. Tên cơ sở dữ liệu:

- Quy tắc:

Tên CSDL phải ngắn gọn, dễ hiểu, phản ánh đúng mục đích của hệ thống.

Sử dụng tiếng Anh, không chứa khoảng trắng, có thể dùng dấu gạch dưới _ để phân tách từ.

Tránh sử dụng ký tự đặc biệt hoặc từ khóa hệ thống của SQL.

- Ví dụ:

warehouse_management (Hệ thống quản lý kho)

inventory_system (Hệ thống quản lý hàng tồn kho)

QLKhoHang (Dùng tiếng Việt không dấu, khó mở rộng)

2.2.4. Tên bảng trong cơ sở dữ liệu:

- Quy tắc:

Tên bảng phải phản ánh đúng nội dung dữ liệu được lưu trữ.

Sử dụng tiếng Anh, không chứa khoảng trắng, dùng dấu gạch dưới _ để phân tách từ.

Tên bảng ở dạng số nhiều để thể hiện chứa nhiều bản ghi.

- Ví dụ:

Đúng: `users`, `order_details`, `products`.

Sai: `User`, `OrderDetails`.

2.2.5. Tên khóa chính và khóa ngoại:

- Khóa chính (Primary Key – PK):

Tên khóa chính thường đặt theo dạng `id_tên_bảng`. Luôn là số nguyên (INT hoặc BIGINT), tự động tăng (AUTO_INCREMENT).

- Ví dụ:

`id_user` (Khóa chính của bảng `users`)

`id_product` (Khóa chính của bảng `products`)

`id_order` (Khóa chính của bảng `orders`)

- Khóa ngoại (Foreign Key – FK):

Tên khóa ngoại thường có dạng `tên_bảng_được_tham_chiếu_id`.

Đảm bảo tuân thủ ràng buộc FOREIGN KEY với bảng cha

2.3. Xác định Mục tiêu chất lượng của dự án

ST T	Tên độ đo	Ý nghĩa	Công thức/cách đo	Mục tiêu
1	Thời gian phản hồi hệ thống	Đảm bảo hệ thống hoạt động mượt mà,	Cách đo: Sử dụng công cụ (Postman, JMeter) hoặc tab Network trên	Thời gian phản hồi trung bình của API

	(Response Time)	giúp thu ngân chốt đơn nhanh và nhà bếp nhận thông tin tức thời, không làm khách hàng phải chờ đợi.	trình duyệt để đo thời gian phản hồi của các API chính.	Đặt món / Thanh toán < 2 giây.
2	Độ sẵn sàng của hệ thống (System Uptime)	Đảm bảo phần mềm hoạt động liên tục, không bị sập nguồn (crash) trong suốt thời gian cửa hàng mở cửa kinh doanh.	Công thức: $\frac{\text{Thời gian hệ thống hoạt động ổn định} / \text{Tổng thời gian mở cửa của quán}}{\text{}} \times 100\%$.	> 99% (Hầu như không gián đoạn)
3	Mật độ lỗi (Defect Density)	Đánh giá chất lượng lập trình của đội Dev và mức độ hiệu quả của Tester, đảm bảo ít lỗi nghiêm trọng bị lọt ra môi trường thực tế.	Công thức: $\frac{\text{Tổng số lỗi (Bugs) phát hiện} / \text{Tổng số tính năng (User Stories) trong 1 Sprint}}{\text{}}$.	< 2 lỗi nghiêm trọng trên mỗi Sprint hoàn thành.

4	Tỷ lệ hoàn thành Sprint (Sprint Completion Rate)	Đánh giá khả năng ước lượng thời gian và năng suất làm việc thực tế của nhóm làm theo mô hình Scrum.	Công thức: $(\text{Số lượng User Story Point đã hoàn thành} / \text{Tổng số Story Point đã cam kết đầu Sprint}) * 100\%$.	Đạt > 85% cho mỗi Sprint.
5	Tính khả dụng (Usability / Training Time)	Đánh giá thiết kế giao diện (UI/UX) có trực quan, dễ dùng hay không, đặc biệt quan trọng với nhân viên thu ngân/bếp thường xuyên luân chuyển.	Cách đo: Đo thời gian trung bình để đào tạo một nhân viên mới sử dụng thành thạo luồng "Tạo đơn -> Thanh toán".	Thời gian học việc < 30 phút / nhân viên.
6	Tính bảo mật và Phân quyền (Security & Authorization)	Đảm bảo an toàn dữ liệu và chống gian lận (VD: nhân viên thu ngân không thể tự ý xóa hóa đơn, nhà bếp không	Cách đo: Kiểm thử phân quyền (Role-based Access Control).	100% các API nghiệp vụ quan trọng (Sửa/Xóa đơn, Báo cáo) phải yêu cầu Token hợp lệ và đúng quyền. Số lượng lỗi hồng

		được xem báo cáo doanh thu).		bảo mật nghiêm trọng = 0.
7	Khả năng bảo trì mã nguồn (Code Maintainability)	Đảm bảo nhóm làm việc chung không bị "xung đột" code, code viết ra thống nhất, sạch sẽ để dễ dàng bảo trì hoặc bàn giao sau này.	Cách đo: Sử dụng công cụ quét mã nguồn tự động (SonarQube hoặc ESLint cho ReactJS / Checkstyle cho Java).	Tỷ lệ trùng lặp mã nguồn (Code duplication) < 5%.

2.4. Xác định tiêu chuẩn Kiểm thử được (Testability) cho dự án

2.4.1. Khái niệm, ý nghĩa của đặc tính Kiểm thử được

Tính kiểm thử được (Testability) là một đặc tính của phần mềm cho biết mức độ dễ dàng mà các kịch bản kiểm thử (test cases) có thể được thiết lập và thực thi để phát hiện lỗi. Một hệ thống có tính kiểm thử cao sẽ cho phép quan sát các trạng thái bên trong và kiểm soát các đầu vào một cách minh bạch.

Ý nghĩa: Giúp đội ngũ Tester dễ dàng xác minh các yêu cầu chức năng (như đặt món, thanh toán) có chạy đúng hay không.

- Giảm thiểu thời gian và công sức trong việc tái hiện các lỗi nghiệp vụ phức tạp.

- Đảm bảo độ tin cậy của hệ thống trước khi triển khai thực tế tại cửa hàng, tránh rủi ro tính sai tiền hoặc mất đơn hàng của khách.

2.4.2. Chi tiết hóa các tiêu chí của đặc tính Kiểm thử được

STT	Tiêu chí	Yêu cầu cụ thể đối với dự án
1	Tính quan sát được (Observability)	Hệ thống phải ghi lại nhật ký (logs) chi tiết cho các giao dịch quan trọng như: Luồng đơn hàng từ Tablet xuống Bếp, các lệnh thanh toán và lịch sử nhập/xuất kho nguyên liệu để Tester có thể đối chiếu dữ liệu khi có lỗi xảy ra
2	Tính kiểm soát được (Controllability)	Cho phép Tester can thiệp vào trạng thái dữ liệu (ví dụ: giả lập tình trạng kho hết hàng hoặc bàn đang có khách) thông qua Database hoặc công cụ quản lý để kiểm tra phản ứng của hệ thống mà không cần chờ đợi điều kiện thực tế.
3	Tính phân rã được (Decomposability)	Nhờ kiến trúc 3 lớp (3-tier) và mô hình MVC, hệ thống phải cho phép kiểm thử độc lập từng phần: Kiểm thử giao diện (Frontend), kiểm thử logic nghiệp vụ (API/Backend) và kiểm thử dữ liệu (Database) mà không nhất thiết phải chạy toàn bộ hệ thống.
4	Tính đơn giản (Simplicity)	Mã nguồn cần tuân thủ đúng quy tắc đặt tên (Naming Convention) đã đề ra (camelCase cho biến, PascalCase cho Class). Các hàm xử lý (như tính tổng tiền hóa đơn) phải đơn giản, rõ ràng để dễ dàng viết Unit Test.

5	Tính cô lập (Isolation)	Mỗi yêu cầu chức năng phải có mã định danh (ID) riêng trong tài liệu SRS (như PB01, PB02). Điều này giúp Tester tạo kịch bản kiểm thử (Test Case) tương ứng cho từng chức năng mà không bị chồng chéo.
---	-------------------------	--

CHƯƠNG 3. XÁC ĐỊNH YÊU CẦU

3.1. Vai trò của việc Xác định yêu cầu đối với dự án phần mềm

Xác định yêu cầu là bước rất quan trọng trong quá trình phát triển phần mềm. Giai đoạn này giúp nhóm phát triển hiểu rõ hệ thống cần thực hiện những chức năng gì, phục vụ cho những đối tượng nào và phải đáp ứng các điều kiện ra sao trong thực tế. Nếu yêu cầu được xác định đầy đủ và chính xác ngay từ đầu, dự án sẽ có cơ sở rõ ràng để thực hiện các bước tiếp theo như phân tích, thiết kế, lập trình và kiểm thử.

Đối với dự án quản lý bán hàng cho cửa hàng đồ ăn nhanh FAF, việc xác định yêu cầu giúp nhóm nắm rõ các nghiệp vụ chính như quản lý menu, đặt món, chế biến, phục vụ, thanh toán và quản lý kho. Nhờ đó, hệ thống được xây dựng sát với thực tế hoạt động của cửa hàng và đáp ứng đúng nhu cầu của người sử dụng.

Ngoài ra, xác định yêu cầu còn giúp giới hạn phạm vi dự án, tránh thiếu sót hoặc phát triển những chức năng không cần thiết. Đây cũng là cơ sở để phân chia công việc, lập kế hoạch thực hiện và kiểm tra chất lượng hệ thống sau này. Nếu yêu cầu không rõ ràng hoặc thiếu chính xác, dự án dễ gặp sai sót, phải sửa đổi nhiều lần, gây tốn thời gian và chi phí.

Xác định yêu cầu giữ vai trò nền tảng đối với toàn bộ dự án phần mềm. Việc thực hiện tốt giai đoạn này sẽ giúp hệ thống quản lý bán hàng FAF được phát triển đúng hướng, đảm bảo chất lượng và phù hợp với thực tế sử dụng.

3.2. Phương pháp đánh giá chất lượng của yêu cầu

3.2.1. Tiêu chí đánh giá yêu cầu phần mềm

Yêu cầu phần mềm đóng vai trò rất quan trọng trong quá trình phát triển hệ thống. Các yêu cầu rõ ràng và chất lượng sẽ giúp nhóm phát triển hiểu đúng nhu cầu của hệ thống, từ đó thiết kế và xây dựng phần mềm một cách chính xác. Ngược lại, nếu yêu cầu không rõ ràng hoặc thiếu sót, hệ thống có thể hoạt động không đúng mục tiêu ban đầu.

Để đánh giá chất lượng của các yêu cầu trong hệ thống **quản lý bán hàng cho cửa hàng đồ ăn nhanh FAF**, nhóm sử dụng một số tiêu chí phổ biến trong kỹ thuật đặc tả yêu cầu phần mềm (Software Requirement Specification – SRS) như sau:

1. Tính rõ ràng (Clarity)

Yêu cầu phải được mô tả rõ ràng, dễ hiểu và tránh gây nhầm lẫn cho người đọc.

Ví dụ tốt:

“Hệ thống cho phép nhân viên thu ngân tạo đơn hàng bằng cách chọn món ăn từ danh sách thực đơn và tự động tính tổng tiền của đơn hàng.”

Ví dụ không rõ ràng:

“Hệ thống hỗ trợ tạo đơn hàng cho khách.”

(Yêu cầu này không mô tả rõ cách tạo đơn hàng hoặc thông tin cần nhập)

Cách đảm bảo tính rõ ràng:

- Sử dụng ngôn ngữ đơn giản, dễ hiểu.
- Tránh các thuật ngữ mơ hồ như “nhanh chóng”, “tốt”, “hiệu quả”.
- Mô tả cụ thể chức năng của hệ thống và đối tượng sử dụng.

2. Tính đầy đủ (Completeness)

Một yêu cầu tốt phải bao quát đầy đủ các thông tin cần thiết liên quan đến chức năng của hệ thống.

Ví dụ tốt:

“Nhân viên có thể tạo đơn hàng, thêm hoặc xóa món ăn trong đơn, hệ thống sẽ tự động tính tổng tiền và lưu thông tin đơn hàng vào cơ sở dữ liệu.”

Ví dụ thiếu sót:

“Hệ thống cho phép tạo đơn hàng.”

(Yêu cầu này chưa đề cập đến cách xử lý món ăn trong đơn hoặc lưu trữ dữ liệu)

Cách đảm bảo tính đầy đủ:

- Xác định đầy đủ các bước thực hiện chức năng.
- Mô tả rõ dữ liệu đầu vào và đầu ra.
- Xem xét các tình huống có thể xảy ra trong quá trình sử dụng hệ thống.

3. Tính nhất quán (Consistency)

Các yêu cầu trong hệ thống không được mâu thuẫn với nhau.

Ví dụ tốt:

Yêu cầu 1: “Hệ thống cho phép nhân viên tạo đơn hàng.”

Yêu cầu 2: “Sau khi đơn hàng được thanh toán, hệ thống sẽ cập nhật doanh thu và trạng thái đơn hàng.”

Hai yêu cầu này hỗ trợ lẫn nhau và không mâu thuẫn.

Ví dụ không nhất quán:

Yêu cầu 1: “Nhân viên phải đăng nhập để sử dụng hệ thống.”

Yêu cầu 2: “Nhân viên có thể sử dụng hệ thống mà không cần đăng nhập.”

Cách đảm bảo tính nhất quán:

- Kiểm tra và rà soát các yêu cầu một cách có hệ thống.
- Tổ chức các buổi thảo luận nhóm để phát hiện và chỉnh sửa các yêu cầu mâu thuẫn.
- Sử dụng các công cụ quản lý yêu cầu để theo dõi các thay đổi.

4. Tính khả thi (Feasibility)

Yêu cầu phải có khả năng thực hiện được với nguồn lực và công nghệ hiện có.

Ví dụ tốt:

“Hệ thống phải lưu trữ thông tin đơn hàng và hiển thị báo cáo doanh thu theo ngày, tháng hoặc năm.”

Ví dụ không khả thi:

“Hệ thống phải xử lý số lượng khách hàng không giới hạn trong cùng một thời điểm.”

Cách đảm bảo tính khả thi:

- Thảo luận với nhóm phát triển để đánh giá khả năng thực hiện yêu cầu.
- Xem xét các ràng buộc về công nghệ, thời gian và nguồn lực.
- So sánh với các hệ thống quản lý bán hàng tương tự.

5. Tính kiểm chứng được (Verifiability)

Một yêu cầu tốt phải có thể kiểm tra được thông qua kiểm thử phần mềm.

Ví dụ tốt:

“Hệ thống phải tính chính xác tổng tiền của đơn hàng dựa trên giá của từng món ăn và số lượng đã chọn.”

Ví dụ không kiểm chứng được:

“Hệ thống phải tính tiền một cách hợp lý.”
(Yêu cầu này không có tiêu chí rõ ràng để kiểm tra)

Cách đảm bảo tính kiểm chứng được:

- Mô tả rõ ràng điều kiện đầu vào và kết quả đầu ra.
- Sử dụng các tiêu chí cụ thể để đánh giá kết quả.
- Xây dựng test case để kiểm tra yêu cầu.

6. Tính ưu tiên (Prioritization)

Các yêu cầu trong hệ thống cần được xác định mức độ ưu tiên để thuận tiện cho quá trình phát triển.

Cách đảm bảo tính ưu tiên:

- Phân loại yêu cầu theo mức độ quan trọng.
- Áp dụng mô hình **MoSCoW**:
- **Must have**: Các chức năng bắt buộc như tạo đơn hàng, thanh toán, quản lý sản phẩm.
- **Should have**: Các chức năng quan trọng nhưng không bắt buộc như báo cáo doanh thu chi tiết.
- **Could have**: Các chức năng bổ sung như thống kê món bán chạy.
- **Won't have**: Các chức năng chưa cần thiết trong phiên bản hiện tại.

7. Tính truy vết được (Traceability)

Yêu cầu phải có khả năng theo dõi và liên kết với các giai đoạn khác của quá trình phát triển phần mềm.

Cách đảm bảo tính truy vết:

- Gán mã định danh (ID) cho từng yêu cầu.
- Lưu trữ yêu cầu trong các tài liệu quản lý dự án.
- Sử dụng bảng **Requirement Traceability Matrix (RTM)** để theo dõi mối liên hệ giữa yêu cầu, thiết kế, lập trình và kiểm thử.

Kết luận

Việc đánh giá yêu cầu phần mềm theo các tiêu chí trên giúp đảm bảo rằng các yêu cầu của hệ thống quản lý bán hàng cho cửa hàng đồ ăn nhanh FAF được xác định rõ ràng, đầy đủ và có thể triển khai trong thực tế. Điều này góp phần giảm thiểu rủi ro trong quá trình phát triển phần mềm, đồng thời đảm bảo hệ thống đáp ứng đúng nhu cầu của người sử dụng.

3.2.2. Phương pháp kiểm tra và xác nhận yêu cầu

Việc kiểm tra và xác nhận yêu cầu là một bước quan trọng trong quá trình phát triển phần mềm nhằm đảm bảo rằng các yêu cầu được xác định đúng với nhu cầu của người dùng, không có sai sót hoặc thiếu sót và có thể triển khai trong thực tế.

Trong hệ thống **quản lý bán hàng cho cửa hàng đồ ăn nhanh FAF**, việc kiểm tra yêu cầu giúp đảm bảo hệ thống đáp ứng đúng các chức năng như quản lý sản phẩm, tạo đơn hàng, thanh toán và thống kê doanh thu.

Dưới đây là một số phương pháp phổ biến được sử dụng để kiểm tra và xác nhận yêu cầu phần mềm.

1. Xét duyệt yêu cầu (Requirement Review)

Xét duyệt yêu cầu là quá trình kiểm tra các yêu cầu một cách có hệ thống với sự tham gia của các bên liên quan như khách hàng, nhà phân tích hệ thống, nhóm phát triển và nhóm kiểm thử.

Mục tiêu của phương pháp này là phát hiện sớm các lỗi, sự thiếu sót hoặc mâu thuẫn trong yêu cầu trước khi hệ thống được thiết kế và lập trình.

Cách thực hiện

- Tổ chức cuộc họp với các bên liên quan để xem xét từng yêu cầu.
- Đánh giá yêu cầu dựa trên các tiêu chí như: rõ ràng, đầy đủ, nhất quán, khả thi và có thể kiểm chứng.
- Ghi nhận các vấn đề phát hiện được và chỉnh sửa yêu cầu nếu cần.

Ví dụ thực tế

Một cuộc họp được tổ chức để xem xét yêu cầu:

“Hệ thống cho phép nhân viên tạo đơn hàng và thanh toán cho khách.”

Trong quá trình thảo luận, có ý kiến cho rằng cần bổ sung thêm chức năng **in hóa đơn cho khách hàng** sau khi thanh toán. Vì vậy yêu cầu được cập nhật lại để phản ánh đúng nhu cầu thực tế của cửa hàng.

2. Tạo mô hình và nguyên mẫu (Prototyping)

Nguyên mẫu (Prototype) giúp trực quan hóa các yêu cầu của hệ thống, từ đó giúp khách hàng và nhóm phát triển dễ hiểu hơn về cách hệ thống sẽ hoạt động.

Phương pháp này thường được sử dụng trong giai đoạn đầu của dự án để kiểm tra tính hợp lý của giao diện và quy trình làm việc.

Cách thực hiện

- Tạo các bản thiết kế giao diện sơ bộ bằng **Wireframe, Mockup hoặc Prototype**.
- Cho người dùng thử nghiệm các bản mô phỏng.
- Thu thập ý kiến phản hồi và điều chỉnh yêu cầu nếu cần.

Ví dụ thực tế

Nhóm phát triển thiết kế giao diện thử nghiệm cho chức năng **tạo đơn hàng** của hệ thống quản lý bán hàng.

Giao diện bao gồm các chức năng:

- Chọn món ăn từ danh sách
- Nhập số lượng
- Hiển thị tổng tiền
- Thanh toán đơn hàng

Sau khi nhân viên cửa hàng thử nghiệm giao diện này, họ đề xuất thêm chức năng **tìm kiếm nhanh món ăn**, từ đó yêu cầu hệ thống được cập nhật lại.

3. Phân tích kịch bản sử dụng (Use Case Analysis)

Phân tích Use Case giúp mô tả cách người dùng tương tác với hệ thống thông qua các kịch bản sử dụng cụ thể.

Thông qua các kịch bản này, nhóm phát triển có thể kiểm tra xem yêu cầu đã đầy đủ hay chưa.

Cách thực hiện

- Xác định các tác nhân (Actor) của hệ thống.
- Xây dựng kịch bản sử dụng (Use Case).
- Kiểm tra các bước thực hiện của hệ thống.

Ví dụ thực tế

Xét yêu cầu: **“Khách hàng có thể tự tạo đơn hàng.”**

Kịch bản sử dụng có thể như sau:

Bước	Mô tả
1	Khách hàng dùng máy của cửa hàng đã đăng nhập vào hệ thống
2	Khách hàng chọn món ăn từ danh sách sản phẩm
3	Khách hàng nhập số lượng món
4	Hệ thống tính tổng tiền của đơn hàng
5	Nhân viên xác nhận thanh toán
6	Hệ thống lưu đơn hàng và in hóa đơn

Thông qua kịch bản này, nhóm phát triển có thể phát hiện thêm các yêu cầu cần bổ sung như **chỉnh sửa đơn hàng hoặc hủy đơn hàng**.

4. Kiểm tra đối chiếu với tài liệu khách hàng (Requirement Validation with Stakeholders)

Phương pháp này giúp đảm bảo rằng các yêu cầu phần mềm phản ánh đúng nhu cầu thực tế của khách hàng.

Cách thực hiện

- So sánh các yêu cầu đã ghi nhận với yêu cầu ban đầu của khách hàng.
- Gửi tài liệu yêu cầu cho khách hàng để xác nhận.
- Thu thập phản hồi và điều chỉnh yêu cầu nếu cần.

Ví dụ thực tế

Ban đầu khách hàng yêu cầu hệ thống **quản lý bán hàng và lưu đơn hàng**.

Sau khi nhóm phát triển gửi tài liệu yêu cầu, khách hàng phản hồi rằng hệ thống cần thêm chức năng **tìm kiếm sản phẩm theo tên hoặc theo mã sản phẩm** để nhân viên có thể tìm món nhanh hơn khi tạo đơn hàng.

Do đó yêu cầu được cập nhật lại để phù hợp hơn với nhu cầu quản lý của cửa hàng.

5. Kiểm tra thông qua các tiêu chí đo lường (Testable Requirements)

Một yêu cầu tốt phải có thể kiểm tra được bằng các tiêu chí đo lường cụ thể.

Cách thực hiện

- Xác định các tiêu chí đo lường cho yêu cầu.
- Xây dựng các test case để kiểm tra yêu cầu.

Ví dụ thực tế

Yêu cầu:

“Hệ thống phải hiển thị danh sách sản phẩm trong vòng **2 giây**.”

Tiêu chí đo lường:

Sử dụng công cụ kiểm thử hiệu năng để đo thời gian phản hồi của hệ thống khi tải danh sách sản phẩm.

6. Mô phỏng tình huống thực tế (Simulation & User Testing)

Phương pháp mô phỏng giúp kiểm tra xem hệ thống có hoạt động đúng trong điều kiện thực tế hay không.

Cách thực hiện

- Tạo dữ liệu thử nghiệm giống dữ liệu thật.
- Cho người dùng thực hiện các thao tác trên hệ thống.
- Ghi nhận lỗi hoặc vấn đề phát sinh.

Ví dụ thực tế

Nhóm phát triển tạo một môi trường thử nghiệm với dữ liệu sản phẩm và đơn hàng.

Khách hàng thực hiện các thao tác như:

- tạo đơn hàng
- thêm món ăn
- thanh toán đơn hàng

Nếu hệ thống xử lý đúng và không xảy ra lỗi thì yêu cầu được xác nhận là phù hợp.

Bảng tổng hợp các phương pháp kiểm tra yêu cầu

Phương pháp	Mục tiêu	Ưu điểm	Nhược điểm
Xét duyệt yêu cầu	Xác minh yêu cầu đúng và hợp lý	Đơn giản, dễ thực hiện	Có thể bỏ sót lỗi nếu không kiểm tra kỹ
Nguyên mẫu	Trực quan hóa yêu cầu	Dễ hiểu và dễ điều chỉnh	Tốn thời gian tạo nguyên mẫu
Phân tích Use Case	Xác định đầy đủ chức năng hệ thống	Mô tả rõ cách hệ thống hoạt động	Cần nhiều thời gian phân tích
Kiểm tra tài liệu	Đảm bảo yêu cầu	Tránh hiểu sai yêu	Phụ thuộc vào tài

khách hàng	đúng nhu cầu	cầu	liệu khách hàng
Kiểm tra đo lường	Xác minh yêu cầu có thể kiểm tra	Có thể viết test case rõ ràng	Khó áp dụng với yêu cầu mơ hồ
Mô phỏng thực tế	Kiểm tra hoạt động của hệ thống	Gần với môi trường thật	Cần môi trường thử nghiệm

Kết luận

Kiểm tra và xác nhận yêu cầu phần mềm là bước quan trọng nhằm đảm bảo hệ thống được xây dựng đúng với nhu cầu của người sử dụng.

Việc áp dụng các phương pháp như xét duyệt yêu cầu, tạo nguyên mẫu, phân tích kịch bản sử dụng, kiểm tra với khách hàng, kiểm tra bằng tiêu chí đo lường và mô phỏng tình huống thực tế giúp phát hiện sớm các vấn đề trong yêu cầu.

Nhờ đó, hệ thống quản lý bán hàng cho cửa hàng đồ ăn nhanh FAF có thể được phát triển chính xác, giảm rủi ro và nâng cao chất lượng phần mềm.

3.2.3. Công cụ hỗ trợ đánh giá yêu cầu phần mềm

Trong quá trình phát triển phần mềm, việc sử dụng các công cụ hỗ trợ giúp việc thu thập, quản lý và đánh giá yêu cầu trở nên hiệu quả hơn. Các công cụ này giúp tăng độ chính xác của yêu cầu, đảm bảo tính nhất quán giữa các giai đoạn phát triển và giảm thiểu các sai sót trong quá trình xây dựng hệ thống.

Đối với hệ thống **quản lý bán hàng cho cửa hàng đồ ăn nhanh FAF**, việc sử dụng các công cụ hỗ trợ giúp nhóm phát triển dễ dàng theo dõi yêu cầu, mô hình hóa hệ thống và kiểm tra tính hợp lý của các chức năng.

Dưới đây là một số nhóm công cụ phổ biến được sử dụng trong quá trình đánh giá yêu cầu phần mềm.

1. Công cụ quản lý yêu cầu (Requirement Management Tools)

Các công cụ này giúp ghi nhận, lưu trữ và theo dõi các yêu cầu phần mềm trong suốt vòng đời của dự án.

1.1 IBM Engineering Requirements Management DOORS

Mô tả

- Là công cụ chuyên dùng để quản lý yêu cầu trong các dự án phần mềm quy mô lớn.
- Cho phép tạo, chỉnh sửa và theo dõi các yêu cầu trong hệ thống.
- Hỗ trợ truy vết yêu cầu từ giai đoạn phân tích cho đến kiểm thử.

Ưu điểm

- Hỗ trợ quản lý yêu cầu cho các dự án lớn và phức tạp.
- Có khả năng tích hợp với nhiều công cụ phát triển phần mềm khác.

Nhược điểm

- Chi phí sử dụng cao.
- Cần người dùng có kinh nghiệm để khai thác hết các chức năng của công cụ.

1.2 Jama Connect

Mô tả

- Là nền tảng quản lý yêu cầu hiện đại giúp theo dõi và quản lý các thay đổi của yêu cầu theo thời gian.
- Hỗ trợ làm việc nhóm và tích hợp với các công cụ như Jira hoặc Azure DevOps.

Ưu điểm

- Giao diện thân thiện và dễ sử dụng.
- Hỗ trợ cộng tác nhóm tốt.

Nhược điểm

- Khả năng tùy chỉnh thấp hơn so với một số công cụ chuyên biệt khác.
- Thường phù hợp với các dự án vừa và nhỏ.

2. Công cụ quản lý vòng đời phát triển ứng dụng (ALM Tools)

Các công cụ ALM hỗ trợ quản lý toàn bộ vòng đời phát triển phần mềm, từ thu thập yêu cầu, thiết kế, lập trình cho đến kiểm thử và triển khai.

2.1 Microsoft Azure DevOps

Mô tả

- Là nền tảng quản lý dự án phần mềm được Microsoft phát triển.
- Hỗ trợ quản lý yêu cầu, theo dõi lỗi, kiểm thử và triển khai phần mềm.

Ưu điểm

- Hỗ trợ cả phương pháp phát triển Agile và Waterfall.
- Phù hợp cho làm việc nhóm và quản lý dự án.

Nhược điểm

- Việc tích hợp với các công cụ ngoài hệ sinh thái Microsoft đôi khi khá phức tạp.

2.2 IBM Engineering Lifecycle Management (ELM)

Mô tả

- Là hệ thống quản lý vòng đời phát triển phần mềm cho các tổ chức lớn.
- Hỗ trợ quản lý yêu cầu, thiết kế, kiểm thử và triển khai trong cùng một môi trường.

Ưu điểm

- Quản lý toàn bộ quy trình phát triển phần mềm.
- Hỗ trợ truy vết yêu cầu mạnh mẽ.

Nhược điểm

- Chi phí triển khai cao.
- Thời gian triển khai tương đối dài.

3. Công cụ hỗ trợ thu thập và mô hình hóa yêu cầu

Nhóm công cụ này giúp biểu diễn yêu cầu dưới dạng sơ đồ hoặc mô hình trực quan để dễ hiểu và dễ phân tích.

3.1 Microsoft Visio

Mô tả

- Là công cụ dùng để vẽ sơ đồ và mô hình hệ thống.
- Hỗ trợ vẽ các loại sơ đồ như UML, sơ đồ quy trình nghiệp vụ và sơ đồ luồng dữ liệu.

Ưu điểm

- Giao diện quen thuộc với người dùng Microsoft.
- Dễ sử dụng trong việc biểu diễn quy trình và mô hình hệ thống.

Nhược điểm

- Không hỗ trợ quản lý yêu cầu mạnh như các công cụ chuyên biệt.

3.2 Lucidchart

Mô tả

- Là công cụ vẽ sơ đồ trực tuyến giúp mô hình hóa yêu cầu phần mềm.
- Hỗ trợ làm việc nhóm trực tiếp trên trình duyệt.

Ưu điểm

- Không cần cài đặt phần mềm.
- Có thể cộng tác nhiều người cùng lúc.

Nhược điểm

- Phiên bản miễn phí có giới hạn một số chức năng.

4. Công cụ theo dõi yêu cầu và quản lý công việc

Các công cụ này giúp theo dõi tiến độ dự án và quản lý các yêu cầu của hệ thống.

4.1 Jira Software

Mô tả

- Là công cụ phổ biến dùng để quản lý yêu cầu và theo dõi lỗi trong phát triển phần mềm.
- Hỗ trợ quản lý backlog, sprint và theo dõi tiến độ theo phương pháp Agile.

Ưu điểm

- Tích hợp tốt với nhiều công cụ phát triển phần mềm khác.
- Phù hợp với các nhóm phát triển Agile.

Nhược điểm

- Việc cấu hình ban đầu có thể phức tạp đối với người mới.

4.2 Trello

Mô tả

- Là công cụ quản lý công việc trực quan dựa trên phương pháp Kanban.
- Cho phép theo dõi tiến độ công việc thông qua các bảng và thẻ.

Ưu điểm

- Giao diện đơn giản và dễ sử dụng.
- Phù hợp với các nhóm nhỏ hoặc dự án đơn giản.

Nhược điểm

- Ít tính năng hơn so với các công cụ quản lý dự án chuyên nghiệp.

5. Công cụ kiểm tra và xác thực yêu cầu

Các công cụ này giúp kiểm tra xem các yêu cầu phần mềm có được thực hiện đúng trong quá trình phát triển hay không.

5.1 Selenium

Mô tả

- Là công cụ kiểm thử tự động cho các ứng dụng web.
- Hỗ trợ kiểm tra giao diện và chức năng của hệ thống trên nhiều trình duyệt.

Ưu điểm

- Giúp tự động hóa quá trình kiểm thử.
- Hỗ trợ nhiều ngôn ngữ lập trình như Java, Python, C#.

Nhược điểm

- Người sử dụng cần có kiến thức lập trình để sử dụng hiệu quả.

5.2 TestRail

Mô tả

- Là công cụ quản lý kiểm thử giúp theo dõi và đánh giá chất lượng yêu cầu phần mềm.
- Hỗ trợ xây dựng test case và quản lý kết quả kiểm thử.

Ưu điểm

- Theo dõi toàn bộ quá trình kiểm thử phần mềm.
- Giúp đánh giá yêu cầu có thể kiểm chứng được hay không.

Nhược điểm

- Cần tích hợp với các công cụ khác để đạt hiệu quả tối đa.

Kết luận

Có nhiều công cụ hỗ trợ đánh giá và quản lý yêu cầu phần mềm trong quá trình phát triển hệ thống. Việc lựa chọn công cụ phù hợp phụ thuộc vào quy mô dự án, ngân sách và nhu cầu của nhóm phát triển.

Đối với dự án **hệ thống quản lý bán hàng cửa hàng FAF**, các công cụ như Visio, Trello hoặc Jira có thể được sử dụng để mô hình hóa yêu cầu, quản lý công việc và theo dõi tiến độ phát triển hệ thống.

3.3. Danh sách các yêu cầu của hệ thống

3.3.1 Yêu cầu chức năng

1. Quản lý menu và đặt món

- Hệ thống hiển thị menu điện tử trên máy tính bảng tại mỗi bàn.
- Khách hàng có thể chọn món và đặt món trực tiếp trên máy tính bảng.
- Hệ thống tự động tạo đơn đặt món khi khách hàng lựa chọn món ăn.
- Hệ thống gửi đơn đặt món đến bộ phận giao dịch và bộ phận chế biến.
- Khách hàng có thể đặt thêm món trong quá trình phục vụ.

2. Quản lý chế biến và phục vụ món

- Hệ thống hiển thị danh sách đơn đặt món cho bộ phận chế biến.
- Bộ phận chế biến có thể cập nhật trạng thái món ăn (đang chế biến / hoàn thành).
- Hệ thống thông báo cho bộ phận phục vụ khi món đã hoàn thành.
- Bộ phận phục vụ xác nhận đã phục vụ món cho khách hàng.
- Hệ thống ghi nhận phản hồi của khách hàng về món ăn.

3. Quản lý thanh toán

- Cho phép khách hàng yêu cầu thanh toán.
- Hệ thống tổng hợp các đơn đặt món của bàn để lập hóa đơn.
- Hệ thống cho phép lập hóa đơn bán hàng theo từng bàn tại cửa hàng, đồng thời hỗ trợ lập hóa đơn đối với các đơn đặt món trực tuyến. Đối với đơn trực tuyến, hệ thống ghi nhận theo bàn số 0 và bổ sung các thông tin liên quan như tên người đặt và số điện thoại.
- Cho phép chỉnh sửa hóa đơn khi có sai sót.
- Hỗ trợ nhiều phương thức thanh toán (tiền mặt, thẻ, ví điện tử, QR).

4. Quản lý nguyên liệu và kho

- Cho phép lập phiếu yêu cầu nguyên liệu.
- Quản lý xuất kho nguyên liệu cho bộ phận chế biến.
- Lưu thông tin nguyên liệu đã sử dụng.
- Quản lý nhập kho nguyên liệu từ nhà cung cấp.
- Lưu thông tin nhà cung cấp và lịch sử nhập hàng.

5. Quản lý kiểm kê kho

- Cho phép kiểm kê nguyên liệu trong kho theo định kỳ.
- Hệ thống lập báo cáo nguyên liệu nhập và sử dụng.
- Hệ thống tạo phiếu yêu cầu bổ sung nguyên liệu khi tồn kho thấp.

3.3.2 Yêu cầu phi chức năng

1. Hiệu năng

- Hệ thống xử lý đơn đặt món nhanh chóng, thời gian phản hồi dưới 2 giây.
- Hệ thống cho phép nhiều bàn đặt món đồng thời mà không làm chậm hệ thống.

2. Tính sẵn sàng

- Hệ thống hoạt động liên tục trong giờ mở cửa của cửa hàng.
- Hệ thống có cơ chế khôi phục dữ liệu khi xảy ra sự cố.

3. Bảo mật và phân quyền

- Hệ thống phân quyền truy cập cho từng bộ phận (thu ngân, bếp, kho...).
- Dữ liệu thanh toán và thông tin khách hàng được bảo vệ khỏi truy cập trái phép.

4. Khả năng sử dụng

- Giao diện trên máy tính bảng dễ sử dụng đối với khách hàng.
- Nhân viên có thể thao tác dễ dàng trên hệ thống quản lý.

5. Khả năng mở rộng

- Hệ thống có thể mở rộng khi số lượng bàn hoặc chi nhánh tăng.

6. Khả năng bảo trì

- Hệ thống dễ dàng cập nhật menu, giá món và nguyên liệu.
- Hệ thống dễ dàng sửa lỗi và nâng cấp trong tương lai.

3.3.3 Ràng buộc hệ thống

1. Ràng buộc về thiết bị

- Hệ thống phải hoạt động trên máy tính bảng đặt tại mỗi bàn để khách hàng đặt món.
- Hệ thống quản lý phải chạy trên máy tính tại quầy thu ngân và khu vực bếp.
- Các thiết bị trong cửa hàng phải kết nối cùng một mạng nội bộ (LAN hoặc WiFi).

2. Ràng buộc về người sử dụng

- Khách hàng chỉ được phép xem menu, đặt món và gửi phản hồi.
- Nhân viên bếp chỉ được phép xem đơn đặt món và cập nhật trạng thái chế biến.
- Thu ngân chỉ được phép quản lý hóa đơn và thực hiện thanh toán.
- Bộ phận quản lý kho chỉ được phép quản lý nhập kho, xuất kho và kiểm kê nguyên liệu.

3. Ràng buộc về dữ liệu

- Mỗi đơn đặt món phải gắn với một bàn cụ thể.
- Mỗi hóa đơn thanh toán phải tương ứng với một hoặc nhiều đơn đặt món của bàn đó.
- Thông tin nguyên liệu trong kho phải được cập nhật khi có nhập hoặc xuất kho.
- Dữ liệu đơn hàng và hóa đơn phải được lưu trữ để phục vụ báo cáo và kiểm tra.

4. Ràng buộc về quy trình nghiệp vụ

- Món ăn chỉ được phục vụ sau khi bộ phận chế biến cập nhật trạng thái hoàn thành.
- Hóa đơn chỉ được lập sau khi khách hàng yêu cầu thanh toán.
- Nguyên liệu chỉ được xuất kho khi có phiếu yêu cầu nguyên liệu hợp lệ.
- Nguyên liệu chỉ được nhập kho sau khi kiểm tra từ nhà cung cấp.

5. Ràng buộc về thời gian

- Hệ thống phải xử lý đơn đặt món và cập nhật trạng thái nhanh để tránh chậm phục vụ.
- Dữ liệu bán hàng phải được tổng hợp vào cuối mỗi ngày.

6. Ràng buộc về bảo mật

- Nhân viên phải đăng nhập hệ thống trước khi sử dụng các chức năng quản lý.
- Chỉ người có quyền mới được sửa hóa đơn hoặc thay đổi dữ liệu kho.

3.4. Kết quả đánh giá chất lượng các yêu cầu

Qua quá trình rà soát, có thể đánh giá tập yêu cầu của hệ thống theo các nội dung sau:

3.4.1. Mức độ phù hợp với kết quả khảo sát và định hướng đề tài

- Tập yêu cầu được xây dựng tương đối sát với thực tế hoạt động của cửa hàng đồ ăn nhanh đã khảo sát.
- Các yêu cầu phản ánh đúng những nhu cầu cốt lõi trong quy trình vận hành của cửa hàng, từ khâu khách hàng gọi món đến khâu chế biến, phục vụ và thanh toán.
- Nội dung yêu cầu bám sát định hướng của đề tài là xây dựng hệ thống hỗ trợ quản lý cửa hàng đồ ăn nhanh theo hướng số hóa quy trình nghiệp vụ.
- Việc xác định yêu cầu dựa trên khảo sát giúp hạn chế tình trạng xây dựng hệ thống xa rời thực tế hoặc chứa các chức năng không thật sự cần thiết.

3.4.2. Mức độ bao quát các chức năng chính của hệ thống

- Tập yêu cầu đã bao quát được phần lớn các nghiệp vụ quan trọng của cửa hàng.
- Các chức năng trọng tâm đã được xác định gồm:
 - hiển thị thực đơn điện tử;
 - cho phép khách hàng chọn món và đặt món tại bàn;
 - lập đơn gọi món;
 - gửi thông tin món xuống bộ phận bếp;
 - cập nhật trạng thái chế biến món ăn;
 - hỗ trợ phục vụ món cho khách;
 - lập và xử lý hóa đơn thanh toán;
 - quản lý nguyên liệu nhập kho, xuất kho;
 - kiểm kê kho nguyên liệu.
- Việc xác định đầy đủ các nhóm chức năng này cho thấy hệ thống được định hình theo hướng phục vụ trực tiếp cho chu trình hoạt động chính của cửa hàng.

3.4.3. Cấu trúc trình bày yêu cầu rõ ràng và hợp lý

- Các yêu cầu được chia thành hai nhóm chính:
 - yêu cầu chức năng;
 - yêu cầu phi chức năng.
- Cách phân nhóm này giúp tài liệu có bố cục rõ ràng, dễ theo dõi và thuận tiện cho việc phân tích.
- Nhóm yêu cầu chức năng thể hiện hệ thống cần thực hiện những công việc gì.
- Nhóm yêu cầu phi chức năng thể hiện các điều kiện, tiêu chuẩn và chất lượng mà hệ thống phải đáp ứng khi vận hành.
- Cấu trúc này giúp người đọc dễ dàng phân biệt giữa “chức năng cần có” và “mức chất lượng cần đạt được”.

3.4.4. Chất lượng của nhóm yêu cầu chức năng

- Nhóm yêu cầu chức năng phản ánh khá đầy đủ các hoạt động nghiệp vụ chủ yếu.
- Các yêu cầu có tính thực tiễn cao, gắn với công việc hằng ngày của từng bộ phận trong cửa hàng.

- Một số chức năng được mô tả khá rõ và dễ hình dung, chẳng hạn:
 - khách hàng xem menu và chọn món trên thiết bị;
 - hệ thống ghi nhận đơn đặt món;
 - bếp nhận thông tin món cần chế biến;
 - bếp cập nhật trạng thái món;
 - phục vụ theo dõi món đã hoàn thành để mang ra bàn;
 - thu ngân thực hiện thanh toán theo bàn và theo đơn đặt online;
 - kho theo dõi nhập, xuất và tồn nguyên liệu.
- Nhìn chung, các yêu cầu chức năng đã hỗ trợ tốt cho bước tiếp theo như phân tích nghiệp vụ, lập sơ đồ chức năng và thiết kế quy trình xử lý.

3.4.5. Chất lượng của nhóm yêu cầu phi chức năng

- Nhóm yêu cầu phi chức năng cho thấy hệ thống không chỉ được quan tâm ở mặt chức năng mà còn ở khả năng vận hành thực tế.
- Các nội dung phi chức năng đã đề cập đến các yếu tố quan trọng như:
 - tốc độ xử lý;
 - tính ổn định;
 - tính bảo mật;
 - khả năng sử dụng;
 - khả năng mở rộng.
- Đây là những yếu tố cần thiết đối với hệ thống quản lý cửa hàng, vì hệ thống phải hoạt động liên tục, dễ sử dụng và hỗ trợ nhiều bộ phận cùng lúc.
- Việc đưa các yêu cầu phi chức năng vào tài liệu từ sớm là điểm tích cực, vì giúp định hướng rõ hơn cho quá trình thiết kế và triển khai sau này.

3.4.6. Mức độ phù hợp với phạm vi hệ thống

- Tập yêu cầu hiện tại tương đối phù hợp với phạm vi đề tài đã xác định.
- Hệ thống được giới hạn trong các nghiệp vụ cốt lõi của cửa hàng đồ ăn nhanh, tránh mở rộng quá mức.
- Trọng tâm của hệ thống tập trung vào:
 - gọi món tại bàn;
 - phối hợp giữa khách hàng, bếp, phục vụ và thu ngân;
 - thanh toán theo bàn;
 - quản lý nguyên liệu và kho.

- Những chức năng ngoài phạm vi như:
 - khách hàng tự tạo tài khoản cá nhân;
 - chương trình tích điểm phức tạp;
 - quản trị chuỗi nhiều chi nhánh ở mức nâng cao;
 - các chức năng phân tích kinh doanh chuyên sâu; hiện chưa đưa vào.
- Việc giới hạn phạm vi như vậy là hợp lý, giúp đề tài tập trung hơn và phù hợp với khả năng thực hiện của nhóm trong thời gian có hạn.

3.4.7. Mức độ rõ ràng và dễ hiểu của yêu cầu

- Phần lớn các yêu cầu được diễn đạt theo hướng tương đối rõ ràng, dễ hiểu.
- Nội dung yêu cầu bám sát các nghiệp vụ quen thuộc nên thuận lợi cho người đọc tiếp cận.
- Nhiều yêu cầu đã thể hiện được:
 - tác nhân liên quan;
 - công việc cần thực hiện;
 - mục đích của chức năng.
- Điều này giúp giảm nguy cơ hiểu sai giữa các thành viên trong nhóm khi chuyển sang giai đoạn phân tích và thiết kế.
- Ngoài ra, việc mô tả các chức năng theo luồng nghiệp vụ thực tế cũng làm tăng tính khả thi khi mô hình hóa bằng BFD, DFD hoặc Use Case.

3.4.8. Mức độ hỗ trợ cho các giai đoạn phát triển tiếp theo

- Tập yêu cầu hiện có là cơ sở tương đối tốt để chuyển sang các bước tiếp theo của quá trình phát triển hệ thống.
- Từ bộ yêu cầu này, nhóm có thể tiếp tục:
 - xây dựng sơ đồ phân rã chức năng;
 - mô hình hóa luồng dữ liệu;
 - xác định tác nhân và ca sử dụng;
 - thiết kế cơ sở dữ liệu;
 - thiết kế giao diện hệ thống.
- Việc các yêu cầu chức năng chính đã được xác định tương đối rõ sẽ giúp giảm thời gian chỉnh sửa lớn ở các giai đoạn sau.

- Đồng thời, bộ yêu cầu cũng tạo nền tảng để kiểm tra tính đầy đủ của hệ thống sau khi xây dựng.

3.4.9. Những điểm còn hạn chế cần hoàn thiện thêm

Mặc dù tập yêu cầu đã có mức độ hoàn thiện tương đối tốt, nhưng vẫn còn một số điểm cần tiếp tục làm rõ:

- Một số yêu cầu mới dừng ở mức định hướng, chưa đủ chi tiết để phục vụ trực tiếp cho thiết kế.
- Cơ chế phân quyền giữa các bộ phận như quản lý, thu ngân, bếp, kho và phục vụ chưa được mô tả cụ thể.
Nội dung về sao lưu và khôi phục dữ liệu mới chỉ được nêu khái quát, chưa thể hiện rõ cách thức xử lý khi có sự cố.
- Các yêu cầu liên quan đến khả năng mở rộng hệ thống khi số lượng người dùng hoặc chi nhánh tăng lên vẫn cần được cụ thể hóa hơn.
- Một số tiêu chí phi chức năng như thời gian phản hồi, mức độ an toàn dữ liệu, hoặc yêu cầu về độ chính xác kiểm kê kho có thể cần được lượng hóa rõ hơn.
- Chưa nêu thật chi tiết các tình huống ngoại lệ như:
 - món bị hủy;
 - nguyên liệu thiếu;
 - khách đổi món;
 - thanh toán thất bại;
 - mất kết nối giữa các bộ phận.
- Nếu bổ sung rõ các nội dung này, tài liệu yêu cầu sẽ chặt chẽ và có giá trị hơn trong kiểm thử và triển khai.

3.4.10. Kết luận đánh giá

- Nhìn chung, tập yêu cầu của hệ thống đã được xây dựng khá phù hợp với mục tiêu đề tài và thực tế vận hành của cửa hàng đồ ăn nhanh.
Bộ yêu cầu đã thể hiện được cả hai khía cạnh quan trọng:
 - hệ thống cần làm gì;
 - hệ thống cần vận hành như thế nào.

- Các chức năng chính đã được xác định tương đối đầy đủ, có tính thực tế và phù hợp với phạm vi đề tài.
- Tuy vẫn còn một số điểm cần chi tiết hóa thêm, nhưng đây không phải là hạn chế lớn.
- Có thể khẳng định rằng tập yêu cầu hiện tại đã đạt mức tương đối tốt và đủ làm cơ sở cho các bước phân tích, thiết kế và xây dựng hệ thống ở giai đoạn tiếp theo.

CHƯƠNG 4. KIẾN TRÚC PHẦN MỀM

4.1. Vai trò của kiến trúc phần mềm với quá trình phát triển dự án phần mềm

Kiến trúc phần mềm giữ vai trò định hướng tổng thể cho quá trình phân tích, thiết kế, xây dựng và bảo trì hệ thống. Đây là cơ sở để xác định cách tổ chức các thành phần chức năng, mối quan hệ giữa các bộ phận và cách dữ liệu được trao đổi trong toàn bộ hệ thống.

Đối với dự án quản lý bán hàng cửa hàng đồ ăn nhanh FAF, kiến trúc phần mềm có ý nghĩa đặc biệt quan trọng vì hệ thống phải hỗ trợ nhiều nghiệp vụ liên thông như gọi món, chế biến, phục vụ, thanh toán, quản lý kho và kiểm kê. Nếu không xác định kiến trúc phù hợp ngay từ đầu, hệ thống dễ rơi vào tình trạng mã nguồn rời rạc, khó mở rộng và khó bảo trì.

Kiến trúc phần mềm giúp nhóm phát triển:

- định hình cấu trúc tổng thể của hệ thống;
- phân chia rõ trách nhiệm giữa các thành phần;
- hỗ trợ làm việc nhóm hiệu quả hơn;
- tạo cơ sở cho việc lựa chọn công nghệ phù hợp;
- giúp kiểm thử, bảo trì và nâng cấp hệ thống thuận lợi hơn trong tương lai.

Ngoài ra, trong phạm vi học thuật, kiến trúc phần mềm còn giúp việc trình bày báo cáo rõ ràng hơn, dễ liên hệ với các nội dung đã phân tích trước đó như yêu cầu chức năng, yêu cầu phi chức năng, user flow, BFD và DFD.

4.2. Một số kiến trúc phần mềm phổ biến

Trong quá trình phát triển phần mềm hiện nay, có nhiều kiểu kiến trúc được sử dụng tùy theo đặc điểm bài toán, quy mô hệ thống và nguồn lực triển khai. Một số kiến trúc phổ biến gồm:

4.2.1. Monolithic Architecture

Monolithic Architecture là kiến trúc trong đó toàn bộ hệ thống được xây dựng và triển khai như một ứng dụng thống nhất. Các chức năng của hệ thống cùng nằm trong một project hoặc một khối triển khai duy nhất.

Ưu điểm:

- dễ phát triển ở giai đoạn đầu;
- dễ triển khai và kiểm thử;
- phù hợp với hệ thống có quy mô vừa và nhỏ;
- thuận lợi cho nhóm phát triển ít người.

Nhược điểm:

- khi hệ thống quá lớn sẽ khó mở rộng;
- nếu tổ chức mã nguồn không tốt sẽ dễ bị phụ thuộc chặt;
- việc bảo trì về lâu dài có thể phức tạp.

4.2.2. Layered Architecture

Layered Architecture là kiến trúc tổ chức hệ thống theo nhiều lớp, mỗi lớp đảm nhiệm một vai trò riêng. Thông thường hệ thống được chia thành lớp giao diện, lớp xử lý nghiệp vụ, lớp truy xuất dữ liệu và lớp cơ sở dữ liệu.

Ưu điểm:

- mã nguồn rõ ràng, có tổ chức;
- dễ bảo trì và dễ kiểm thử;
- giảm sự phụ thuộc lẫn lộn giữa các chức năng;
- phù hợp với các hệ thống quản lý nghiệp vụ.

Nhược điểm:

- nếu chia lớp không hợp lý có thể gây dư thừa;
- tăng số lượng file và cấu trúc dự án.

4.2.3. Microservices Architecture

Microservices là kiến trúc chia hệ thống thành nhiều dịch vụ nhỏ, độc lập, mỗi dịch vụ phụ trách một nhóm chức năng riêng và giao tiếp với nhau qua API.

Ưu điểm:

- dễ mở rộng từng dịch vụ độc lập;
- phù hợp với hệ thống lớn;
- hỗ trợ triển khai phân tán.

Nhược điểm:

- phức tạp trong phát triển và vận hành;
- khó đồng bộ dữ liệu;
- yêu cầu kinh nghiệm kỹ thuật cao;
- không phù hợp với nhóm nhỏ hoặc đề tài có phạm vi vừa.

4.2.4. Three-tier Architecture

Kiến trúc ba lớp là một dạng triển khai đơn giản và phổ biến, gồm:

- Presentation Layer;
- Business Layer;
- Data Layer.

Kiến trúc này có nhiều điểm tương đồng với Layered Architecture và thường được áp dụng trong các hệ thống quản lý thông tin.

4.3. Kiến trúc tổng thể của hệ thống

Hệ thống quản lý bán hàng cửa hàng đồ ăn nhanh FAF được xây dựng theo mô hình Client–Server kết hợp với kiến trúc phân lớp theo dạng 3 lớp (Three-tier Architecture). Việc lựa chọn kiến trúc này phù hợp với đặc điểm của hệ thống vì có nhiều nhóm người dùng truy cập từ các thiết bị khác nhau như khách hàng trên

máy tính bảng, nhân viên bếp, thu ngân và quản lý. Đồng thời, hệ thống cần xử lý tập trung dữ liệu về đơn hàng, hóa đơn, trạng thái món ăn, kho nguyên liệu và báo cáo doanh thu để bảo đảm tính đồng bộ và chính xác.

4.3.1. Mô hình Client–Server

Trong mô hình Client–Server, hệ thống được chia thành hai phía chính là Client và Server.

- Client là các thiết bị hoặc giao diện mà người dùng trực tiếp thao tác, bao gồm:
 - máy tính bảng tại bàn để khách hàng xem menu và đặt món;
 - màn hình hoặc máy tính tại khu vực bếp để theo dõi và cập nhật trạng thái chế biến;
 - máy tính tại quầy thu ngân để lập hóa đơn và thanh toán;
 - giao diện dành cho quản lý để theo dõi báo cáo và tình hình hoạt động của cửa hàng.
- Server là nơi tiếp nhận yêu cầu từ các client, xử lý nghiệp vụ, truy xuất dữ liệu và trả kết quả về cho người dùng.

Khi khách hàng chọn món trên tablet, yêu cầu sẽ được gửi đến server để xử lý. Sau đó server cập nhật đơn hàng, chuyển thông tin đến bộ phận bếp, đồng thời lưu dữ liệu để phục vụ thanh toán, quản lý kho và báo cáo. Cách tổ chức này giúp các bộ phận trong cửa hàng làm việc đồng bộ với nhau, giảm sai sót và nâng cao hiệu quả vận hành.

4.3.2. Kiến trúc phân lớp (Layered Architecture)

Bên trong mô hình Client–Server, hệ thống được tổ chức theo kiến trúc phân lớp 3 tầng nhằm tách biệt rõ trách nhiệm giữa các thành phần. Cách tổ chức này giúp hệ thống dễ phát triển, dễ kiểm thử, dễ bảo trì và thuận lợi cho việc nâng cấp sau này. Kiến trúc ba lớp cũng là dạng phù hợp với các hệ thống quản lý thông tin như hệ thống FAF.

1. Presentation Layer (Lớp giao diện)

Đây là lớp mà người dùng trực tiếp tương tác. Lớp này có nhiệm vụ hiển thị thông tin và tiếp nhận thao tác từ người dùng như xem menu, chọn món, cập nhật trạng thái món ăn, thanh toán hoặc xem báo cáo.

Ví dụ:

- khách hàng thao tác trên tablet để đặt món;
- nhân viên bếp xem danh sách món cần chế biến;
- thu ngân thực hiện lập hóa đơn và thanh toán;
- quản lý theo dõi doanh thu và tồn kho.

2. Business Logic Layer (Lớp xử lý nghiệp vụ)

Lớp này chịu trách nhiệm xử lý các quy tắc nghiệp vụ cốt lõi của hệ thống.

Ví dụ:

- tạo và cập nhật đơn hàng;
- tính tổng tiền hóa đơn;
- xử lý trạng thái chế biến món ăn;
- quản lý nhập kho, xuất kho và kiểm kê;
- tổng hợp báo cáo doanh thu và thống kê món bán chạy.

Đây là lớp trung tâm quyết định cách hệ thống vận hành theo đúng yêu cầu nghiệp vụ của cửa hàng.

3. Database Layer (Lớp cơ sở dữ liệu)

Lớp này đảm nhiệm cả hai chức năng: truy xuất dữ liệu và lưu trữ dữ liệu. Lớp dữ liệu thực hiện kết nối với cơ sở dữ liệu, xử lý các thao tác thêm, sửa, xóa, tìm kiếm và cập nhật thông tin. Đồng thời, đây cũng là nơi lưu trữ toàn bộ dữ liệu của hệ thống như:

- danh sách món ăn;
- đơn hàng;
- hóa đơn;
- trạng thái chế biến;
- nguyên liệu trong kho;
- thông tin nhà cung cấp;
- dữ liệu báo cáo và thống kê.

Việc gộp phần truy xuất dữ liệu và cơ sở dữ liệu vào cùng một lớp giúp mô hình kiến trúc gọn hơn, dễ trình bày hơn và phù hợp với phạm vi đề tài.

4.3.3. Lý do lựa chọn kiến trúc này cho hệ thống FAF

Nhóm lựa chọn mô hình Client–Server kết hợp với kiến trúc phân lớp vì các lý do sau:

- phù hợp với hệ thống có nhiều thiết bị và nhiều nhóm người dùng;
- dữ liệu được quản lý tập trung, đồng bộ giữa khách hàng, bếp, thu ngân và kho;
- dễ phân chia chức năng theo từng lớp;
- thuận tiện cho việc phát triển, kiểm thử và bảo trì;
- phù hợp với phạm vi đề tài và năng lực triển khai của nhóm.

So với các kiến trúc phức tạp hơn như Microservices, mô hình này đơn giản hơn trong phát triển và triển khai nhưng vẫn đáp ứng tốt nhu cầu của hệ thống quản lý bán hàng tại cửa hàng đồ ăn nhanh. Đồng thời, so với cách tổ chức mã nguồn không có phân lớp, kiến trúc phân lớp giúp hệ thống rõ ràng hơn, tránh phụ thuộc chồng chéo giữa giao diện, xử lý nghiệp vụ và dữ liệu.

4.3.4. Kết luận

Hệ thống FAF được định hướng xây dựng theo mô hình Client–Server để tổ chức việc giao tiếp giữa các thiết bị người dùng và máy chủ xử lý trung tâm, đồng thời áp dụng kiến trúc phân lớp để chia hệ thống thành các phần rõ ràng về chức năng và trách nhiệm. Lựa chọn này giúp hệ thống vừa phù hợp với bài toán quản lý bán hàng thực tế của cửa hàng đồ ăn nhanh, vừa hỗ trợ tốt cho quá trình phát triển, kiểm thử, bảo trì và mở rộng trong tương lai.

4.4. Lựa chọn công nghệ phát triển cho hệ thống

Việc lựa chọn công nghệ phát triển cho hệ thống quản lý bán hàng cửa hàng đồ ăn nhanh FAF cần bảo đảm phù hợp với phạm vi đề tài, đáp ứng được yêu cầu nghiệp vụ và thuận tiện cho quá trình xây dựng, kiểm thử và bảo trì. Do hệ thống được thiết kế theo mô hình Client–Server kết hợp với kiến trúc phân lớp 3 tầng, công

nghe được lựa chọn cũng được phân theo từng tầng của hệ thống gồm: lớp giao diện, lớp xử lý nghiệp vụ và lớp dữ liệu.

4.4.1. Công nghệ cho lớp giao diện (Presentation Layer)

Đối với lớp giao diện, hệ thống lựa chọn các công nghệ web như HTML, CSS, JavaScript kết hợp với ReactJS để xây dựng giao diện cho các nhóm người dùng khác nhau như khách hàng, nhân viên bếp, thu ngân và quản lý. ReactJS phù hợp vì cho phép xây dựng giao diện dạng component, dễ tái sử dụng và thuận tiện khi phát triển nhiều màn hình chức năng khác nhau trong cùng một hệ thống.

Việc sử dụng ReactJS mang lại một số lợi ích như:

- giao diện phản hồi nhanh;
- dễ tổ chức các thành phần hiển thị;
- thuận lợi trong việc mở rộng thêm chức năng mới;
- phù hợp với các thiết bị như tablet, máy tính quầy thu ngân và màn hình bếp.

Nhờ đó, lớp giao diện của hệ thống có thể đáp ứng tốt yêu cầu hiển thị menu, đặt món, cập nhật trạng thái món ăn, thanh toán và xem báo cáo.

4.4.2. Công nghệ cho lớp xử lý nghiệp vụ (Business Logic Layer)

- Đối với tầng xử lý nghiệp vụ, nhóm lựa chọn sử dụng Java Spring Boot để xây dựng phần backend của hệ thống. Đây là nền tảng phù hợp đối với các hệ thống quản lý do có khả năng hỗ trợ tốt trong việc xây dựng API, tổ chức mã nguồn theo cấu trúc rõ ràng và triển khai các chức năng nghiệp vụ một cách ổn định. Trong hệ thống FAF, tầng xử lý nghiệp vụ giữ vai trò trung tâm, đảm nhiệm việc tiếp nhận yêu cầu từ phía giao diện, xử lý logic đặt món, lập hóa đơn, thanh toán, cập nhật trạng thái chế biến, quản lý kho nguyên liệu và tổng hợp dữ liệu phục vụ báo cáo.
- Việc sử dụng Spring Boot giúp nhóm phát triển hệ thống theo hướng chặt chẽ và dễ kiểm soát hơn, đặc biệt trong bối cảnh có nhiều chức năng liên quan chặt chẽ với nhau. Bên cạnh đó, công nghệ này cũng hỗ trợ tốt việc xây dựng các dịch vụ theo mô hình RESTful API, phù hợp với kiến trúc Client–Server mà hệ thống đang áp dụng. Nhờ đó, tầng xử lý nghiệp vụ có thể đóng vai trò cầu nối hiệu quả giữa giao diện người dùng và tầng dữ liệu,

đồng thời bảo đảm các quy tắc nghiệp vụ được thực hiện nhất quán trong toàn hệ thống.

4.4.3. Công nghệ cho lớp cơ sở dữ liệu (Database Layer)

Đối với lớp dữ liệu, hệ thống sử dụng hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) để lưu trữ tập trung dữ liệu của toàn hệ thống. Trong phạm vi triển khai đề tài, nhóm có thể sử dụng MySQL để quản lý dữ liệu.

Lớp dữ liệu có nhiệm vụ:

- lưu trữ thông tin món ăn;
- lưu trữ đơn hàng và hóa đơn;
- quản lý dữ liệu kho nguyên liệu;
- lưu thông tin nhà cung cấp;
- hỗ trợ truy xuất dữ liệu cho báo cáo và thống kê.

Việc lựa chọn cơ sở dữ liệu quan hệ là phù hợp vì dữ liệu của hệ thống có cấu trúc rõ ràng, có nhiều mối liên hệ giữa các bảng như bàn ăn, đơn hàng, món ăn, hóa đơn, nguyên liệu và nhà cung cấp. Điều này giúp việc lưu trữ, truy vấn và bảo đảm tính nhất quán dữ liệu được thực hiện hiệu quả.

4.4.4. Công nghệ giao tiếp giữa các tầng

Trong mô hình Client–Server, việc giao tiếp giữa các tầng của hệ thống được thực hiện thông qua API. Các thiết bị phía người dùng như máy tính bảng tại bàn, máy tính ở quầy thu ngân, thiết bị tại khu vực bếp hoặc giao diện quản lý đóng vai trò là các client gửi yêu cầu đến server. Khi người dùng thực hiện các thao tác như đặt món, cập nhật trạng thái món ăn hoặc thanh toán, yêu cầu sẽ được gửi đến tầng xử lý nghiệp vụ để tiếp nhận và xử lý, sau đó dữ liệu liên quan sẽ được truy xuất hoặc cập nhật tại tầng dữ liệu trước khi phản hồi kết quả trở lại giao diện.

Cơ chế giao tiếp này giúp các bộ phận trong cửa hàng có thể làm việc trên cùng một hệ thống dữ liệu nhưng vẫn bảo đảm sự phân tách rõ ràng giữa giao diện, xử lý nghiệp vụ và dữ liệu. Đồng thời, nó cũng giúp hệ thống dễ mở rộng hơn trong

tương lai khi cần bổ sung thêm thiết bị sử dụng hoặc tích hợp thêm các chức năng mới.

4.4.5. Đánh giá mức độ phù hợp của bộ công nghệ

Nhìn chung, bộ công nghệ được lựa chọn cho hệ thống FAF gồm ReactJS ở tầng giao diện, Java Spring Boot ở tầng xử lý nghiệp vụ và hệ quản trị cơ sở dữ liệu quan hệ ở tầng dữ liệu là phù hợp với đặc điểm của bài toán quản lý bán hàng tại cửa hàng đồ ăn nhanh. Các công nghệ này không chỉ đáp ứng được yêu cầu chức năng của hệ thống mà còn bảo đảm tính khả thi trong triển khai, thuận lợi trong quá trình làm việc nhóm và hỗ trợ tốt cho công tác kiểm thử, bảo trì cũng như mở rộng hệ thống sau này.

Với phạm vi của một đề tài học phần, đây là lựa chọn công nghệ hợp lý, vừa đủ hiện đại để đáp ứng yêu cầu kỹ thuật, vừa đủ thực tiễn để nhóm có thể triển khai và hoàn thiện hệ thống theo đúng định hướng đã đặt ra.

4.5. Kiến trúc phần mềm của hệ thống

Kiến trúc phần mềm của hệ thống quản lý bán hàng cửa hàng đồ ăn nhanh FAF được lựa chọn theo hướng Client–Server kết hợp với kiến trúc phân lớp 3 tầng. Đây là mô hình phù hợp với hệ thống có nhiều nhóm người dùng và nhiều điểm truy cập khác nhau, đồng thời vẫn bảo đảm dữ liệu được quản lý tập trung và xử lý thống nhất.

4.5.1. Mô hình Client–Server

Trong kiến trúc này, các thiết bị phía người dùng đóng vai trò **client**, còn hệ thống xử lý trung tâm đóng vai trò **server**.

Phía client bao gồm:

- máy tính bảng cho khách hàng đặt món;
- thiết bị tại bếp để nhận và cập nhật trạng thái món;
- máy tính thu ngân để lập hóa đơn và thanh toán;
- giao diện quản lý để theo dõi báo cáo và tồn kho.

Phía server có nhiệm vụ:

- tiếp nhận yêu cầu từ client;
- xử lý nghiệp vụ;
- quản lý dữ liệu tập trung;
- trả kết quả về cho từng bộ phận sử dụng.

Mô hình này giúp mọi thao tác trong cửa hàng được liên kết với nhau theo thời gian thực, từ gọi món, chế biến, phục vụ đến thanh toán và cập nhật kho.

4.5.2. Chi tiết các tầng trong kiến trúc phân lớp

Để bảo đảm hệ thống rõ ràng và dễ bảo trì, phần mềm được tổ chức thành 3 lớp chức năng như sau:

- **Presentation Layer (Lớp giao diện)**

Lớp này chịu trách nhiệm hiển thị dữ liệu và tiếp nhận tương tác từ người dùng. Mỗi nhóm người dùng sẽ có giao diện phù hợp với nhiệm vụ của mình, như khách hàng đặt món, bếp cập nhật trạng thái món, thu ngân thanh toán và quản lý xem báo cáo.

- **Business Logic Layer (Lớp xử lý nghiệp vụ)**

Lớp này chứa toàn bộ quy tắc nghiệp vụ chính của hệ thống FAF, bao gồm quản lý đặt món, xử lý hóa đơn, cập nhật tình trạng món ăn, quản lý kho nguyên liệu, tính doanh thu và lập báo cáo. Đây là lớp giữ vai trò trung tâm trong việc điều phối hoạt động của toàn hệ thống.

- **Database Layer (Lớp cơ sở dữ liệu)**

Lớp này chịu trách nhiệm kết nối với cơ sở dữ liệu và xử lý toàn bộ thao tác liên quan đến dữ liệu như thêm, sửa, xóa, tìm kiếm và lưu trữ thông tin. Tất cả dữ liệu về món ăn, đơn hàng, hóa đơn, nguyên liệu, nhà cung cấp và báo cáo đều được quản lý tại lớp này. Việc gộp phần truy xuất dữ liệu và phần lưu trữ dữ liệu vào cùng một lớp giúp hệ thống gọn hơn, dễ trình bày hơn và phù hợp với phạm vi đồ án học phần.

4.5.3. Luồng xử lý dữ liệu (Interaction Flow)

Khi một hành động xảy ra, ví dụ khách hàng nhấn “Đặt món”, dữ liệu sẽ được xử lý qua các lớp theo trình tự sau:

- **Lớp giao diện (Presentation Layer):** Nhận thông tin món ăn, số lượng và yêu cầu từ khách hàng, sau đó gửi yêu cầu đến lớp xử lý nghiệp vụ thông qua API.
- **Lớp xử lý nghiệp vụ (Business Logic Layer):** Tiếp nhận dữ liệu từ lớp giao diện, kiểm tra tính hợp lệ của đơn hàng, tính tổng tiền, xử lý các quy tắc nghiệp vụ liên quan và chuẩn bị dữ liệu để lưu trữ.
- **Lớp cơ sở dữ liệu (Database Layer):** Thực hiện các thao tác truy xuất và lưu trữ dữ liệu vào cơ sở dữ liệu, bao gồm lưu đơn hàng, cập nhật trạng thái đơn và phản hồi kết quả cho lớp nghiệp vụ.

Sau khi dữ liệu được lưu thành công, kết quả sẽ được trả ngược lên lớp nghiệp vụ, rồi chuyển đến lớp giao diện để thông báo cho khách hàng. Đồng thời, hệ thống cũng cập nhật thông tin đơn hàng cho bộ phận bếp và các bộ phận liên quan.

4.5.4. Lợi ích đối với dự án FAF

Việc áp dụng mô hình Client–Server kết hợp với kiến trúc phân lớp 3 tầng mang lại nhiều lợi ích cho dự án FAF như sau:

- **Hỗ trợ phối hợp làm việc nhóm:** Các thành viên có thể phân chia công việc theo từng phần như giao diện, xử lý nghiệp vụ và dữ liệu, từ đó giảm xung đột khi phát triển hệ thống.
- **Dễ bảo trì và kiểm thử:** Việc tách riêng lớp xử lý nghiệp vụ giúp nhóm dễ dàng kiểm thử chức năng, sửa lỗi và nâng cấp hệ thống mà không ảnh hưởng nhiều đến các phần khác.
- **Tăng tính bảo mật:** Người dùng không truy cập trực tiếp vào cơ sở dữ liệu mà phải thông qua lớp xử lý nghiệp vụ và lớp dữ liệu, từ đó giúp bảo vệ dữ liệu kinh doanh tốt hơn.
- **Dễ mở rộng hệ thống:** Khi cửa hàng mở rộng thêm số lượng bàn, người dùng hoặc chức năng mới, hệ thống vẫn có thể nâng cấp thuận lợi nhờ cấu trúc phân lớp rõ ràng.

- **Đảm bảo tính đồng bộ dữ liệu:** Mọi dữ liệu đều được xử lý tập trung qua server, giúp thông tin giữa khách hàng, bếp, thu ngân và quản lý luôn thống nhất.

4.6. Cơ sở dữ liệu mức vật lý của hệ thống

Cơ sở dữ liệu mức vật lý mô tả cách dữ liệu được tổ chức, lưu trữ và truy xuất trong hệ quản trị cơ sở dữ liệu, đồng thời phản ánh cách hệ thống backend tương tác với dữ liệu trong quá trình vận hành. Đối với hệ thống quản lý cửa hàng đồ ăn nhanh FAF, cơ sở dữ liệu giữ vai trò nền tảng, đảm bảo lưu trữ thông tin và hỗ trợ các tiến trình nghiệp vụ như gọi món, chế biến, thanh toán và quản lý kho.

Hệ thống sử dụng cơ sở dữ liệu quan hệ nhằm đảm bảo tính chặt chẽ trong tổ chức dữ liệu và mối liên kết giữa các thực thể. Bên cạnh đó, cơ sở dữ liệu được tích hợp với backend xây dựng bằng Spring Boot, qua đó toàn bộ thao tác truy xuất và xử lý dữ liệu được thực hiện tập trung, góp phần đảm bảo tính nhất quán và ổn định của hệ thống.

4.6.1. Hệ quản trị cơ sở dữ liệu

Hệ thống FAF lựa chọn MySQL làm hệ quản trị cơ sở dữ liệu. Đây là hệ quản trị phổ biến, phù hợp với các hệ thống ứng dụng web và hỗ trợ hiệu quả mô hình dữ liệu quan hệ.

Việc sử dụng MySQL mang lại các lợi ích sau:

- hỗ trợ quản lý dữ liệu có cấu trúc và nhiều mối quan hệ
- đảm bảo tính toàn vẹn dữ liệu thông qua cơ chế transaction
- hỗ trợ khóa chính và khóa ngoại
- dễ triển khai và phù hợp với quy mô của hệ thống

Cơ sở dữ liệu được kết nối với backend thông qua Spring Boot. Backend đóng vai trò tiếp nhận yêu cầu từ giao diện, xử lý logic nghiệp vụ và thực hiện các thao tác truy xuất dữ liệu.

4.6.2. Mô hình lưu trữ dữ liệu

Dữ liệu trong hệ thống được tổ chức theo mô hình cơ sở dữ liệu quan hệ, trong đó mỗi bảng biểu diễn một thực thể hoặc một nghiệp vụ cụ thể.

Các nhóm dữ liệu chính bao gồm:

- nhóm người dùng: quản lý tài khoản và phân quyền
- nhóm thực đơn: quản lý danh mục và món ăn
- nhóm nguyên liệu: quản lý tồn kho và định lượng
- nhóm đơn hàng: quản lý thông tin gọi món
- nhóm hóa đơn: quản lý thanh toán
- nhóm kho: quản lý nhập nguyên liệu

Trong kiến trúc hệ thống, việc truy xuất dữ liệu không được thực hiện trực tiếp từ phía giao diện mà thông qua backend. Luồng xử lý được tổ chức theo mô hình phân tầng, trong đó backend chịu trách nhiệm trung gian giữa giao diện và cơ sở dữ liệu.

4.6.3. Cấu trúc các bảng chính

Cơ sở dữ liệu được thiết kế theo các quy tắc đặt tên thống nhất nhằm đảm bảo tính rõ ràng và dễ bảo trì:

- tên bảng sử dụng tiếng Anh, dạng snake_case và số nhiều
- khóa chính có dạng **id_XXX**
- khóa ngoại có dạng **XXX_id**
- hạn chế sử dụng viết tắt không rõ nghĩa

a) Bảng người dùng và phân quyền

Bảng **roles**

Gồm các thành phần:

- **id_role**: mã vai trò
- **role_name**: tên vai trò

Bảng `users`

Gồm các thành phần:

- `id_user`: mã người dùng
- `username`: tên đăng nhập
- `password_hash`: mật khẩu đã mã hóa
- `full_name`: họ tên người dùng
- `role_id`: mã vai trò (liên kết bảng roles)

```
create table roles (  
    id_role nvarchar(10) primary key,  
    role_name nvarchar(50) not null  
);  
  
create table users (  
    id_user nvarchar(10) primary key,  
    username nvarchar(50) not null unique,  
    password_hash nvarchar(255) not null,  
    full_name nvarchar(100),  
    role_id nvarchar(10),  
    foreign key (role_id) references roles(id_role)  
);
```

b) Bảng món ăn và danh mục món ăn

Bảng `food_categories`

Gồm các thành phần:

- `id_category`: mã danh mục
- `category_name`: tên danh mục

Bảng **foods**

Gồm các thành phần:

- **id_food**: mã món ăn
- **food_name**: tên món ăn
- **category_id**: mã danh mục
- **unit_price**: đơn giá bán

```
create table food_categories (  
    id_category nvarchar(10) primary key,  
    category_name nvarchar(50)  
);  
  
create table foods (  
    id_food nvarchar(10) primary key,  
    food_name nvarchar(100),  
    category_id nvarchar(10),  
    unit_price decimal(18,2),  
    foreign key (category_id) references food_categories(id_category)  
);
```

c) Bảng nguyên liệu và định lượng nguyên liệu

Bảng **ingredients**

Gồm các thành phần:

- **id_ingredient**: mã nguyên liệu
- **ingredient_name**: tên nguyên liệu
- **unit**: đơn vị tính
- **quantity_stock**: số lượng tồn kho

Bảng `food_ingredients`

Gồm các thành phần:

- `food_id`: mã món ăn
- `ingredient_id`: mã nguyên liệu
- `quantity_used`: số lượng nguyên liệu sử dụng

```
create table ingredients (  
    id_ingredient nvarchar(10) primary key,  
    ingredient_name nvarchar(100),  
    unit nvarchar(20),  
    quantity_stock decimal(18,2)  
);  
  
create table food_ingredients (  
    food_id nvarchar(10),  
    ingredient_id nvarchar(10),  
    quantity_used decimal(10,2),  
    primary key (food_id, ingredient_id)  
);
```

d) Bảng đơn đặt món

Bảng `orders`

Gồm các thành phần:

- `id_order`: mã đơn hàng
- `table_number`: số bàn
- `customer_name`: tên khách hàng

- **order_time**: thời gian gọi món
- **status**: trạng thái đơn hàng
- **created_by**: mã người tạo đơn

```
create table orders (
  id_order nvarchar(10) primary key,
  table_number int,
  customer_name nvarchar(100),
  order_time datetime default current_timestamp,
  status nvarchar(30),
  created_by nvarchar(10),
  foreign key (created_by) references users(id_user)
);
```

e) Bảng chi tiết đơn đặt món

Bảng **order_details**

Gồm các thành phần:

- **order_id**: mã đơn hàng
- **food_id**: mã món ăn
- **quantity**: số lượng món
- **unit_price**: đơn giá tại thời điểm đặt
- **status**: trạng thái phục vụ

```
create table order_details (
  order_id nvarchar(10),
  food_id nvarchar(10),
  quantity int,
  unit_price decimal(18,2),
  status nvarchar(10),
  primary key (order_id, food_id),
  foreign key (order_id) references orders(id_order),
  foreign key (food_id) references foods(id_food)
);
```

f) Bảng hóa đơn bán

Bảng **sales_invoices**

Gồm các thành phần:

- **id_invoice**: mã hóa đơn
- **order_id**: mã đơn hàng
- **customer_phone**: số điện thoại khách hàng
- **payment_date**: thời gian thanh toán
- **payment_method**: phương thức thanh toán
- **total_amount**: tổng số tiền thanh toán

```
create table sales_invoices (  
  id_invoice nvarchar(10) primary key,  
  order_id nvarchar(10),  
  customer_phone nvarchar(15),  
  payment_date datetime default current_timestamp,  
  payment_method nvarchar(50),  
  total_amount decimal(18,2),  
  foreign key (order_id) references orders(id_order)  
);
```

g) Bảng nhập kho

Bảng **stock_receipts**

Gồm các thành phần:

- **id_receipt**: mã phiếu nhập kho
- **receipt_date**: ngày nhập kho
- **supplier_name**: tên nhà cung cấp
- **status**: trạng thái phiếu nhập
- **created_by**: mã người tạo phiếu

```
create table stock_receipts (  
  id_receipt nvarchar(10) primary key,
```

```
receipt_date date default (current_date),
supplier_name nvarchar(100),
status nvarchar(20),
created_by nvarchar(10),
foreign key (created_by) references users(id_user)
);
```

h) Bảng chi tiết nhập kho

Bảng `stock_receipt_details`

Gồm các thành phần:

- `receipt_id`: mã phiếu nhập kho
- `ingredient_id`: mã nguyên liệu
- `quantity_import`: số lượng nhập
- `import_price`: đơn giá nhập

```
create table stock_receipt_details (
receipt_id nvarchar(10),
ingredient_id nvarchar(10),
quantity_import decimal(18,2),
import_price decimal(18,2),
primary key (receipt_id, ingredient_id),
foreign key (receipt_id) references stock_receipts(id_receipt),
foreign key (ingredient_id) references ingredients(id_ingredient)
);
```

4.6.4. Tối ưu hóa truy vấn

Để nâng cao hiệu suất xử lý dữ liệu, hệ thống áp dụng các biện pháp tối ưu ở mức cơ sở dữ liệu như:

- xây dựng chỉ mục cho các cột thường xuyên được truy vấn
- thiết kế khóa chính và khóa ngoại hợp lý

- hạn chế truy vấn dư thừa

Ngoài ra, backend có trách nhiệm kiểm soát luồng truy vấn và tổ chức thao tác dữ liệu một cách hợp lý nhằm đảm bảo hiệu năng của hệ thống trong quá trình vận hành.

4.6.5. Sao lưu và phục hồi dữ liệu

Hệ thống thực hiện sao lưu dữ liệu định kỳ nhằm đảm bảo an toàn thông tin và khả năng phục hồi khi xảy ra sự cố.

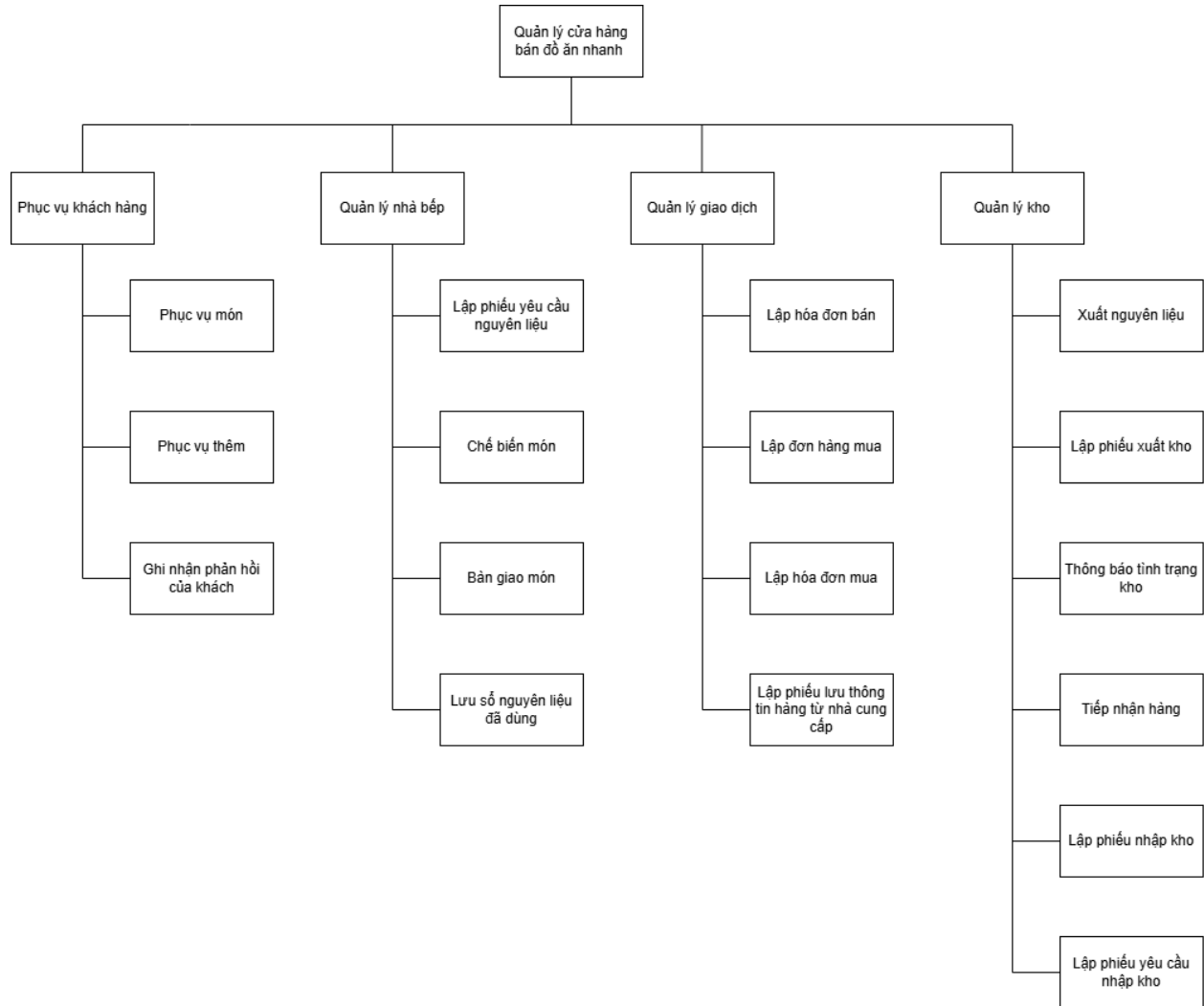
Các biện pháp được áp dụng bao gồm:

- sao lưu toàn bộ cơ sở dữ liệu theo chu kỳ
- lưu trữ bản sao tại nhiều vị trí
- kiểm tra định kỳ khả năng phục hồi dữ liệu

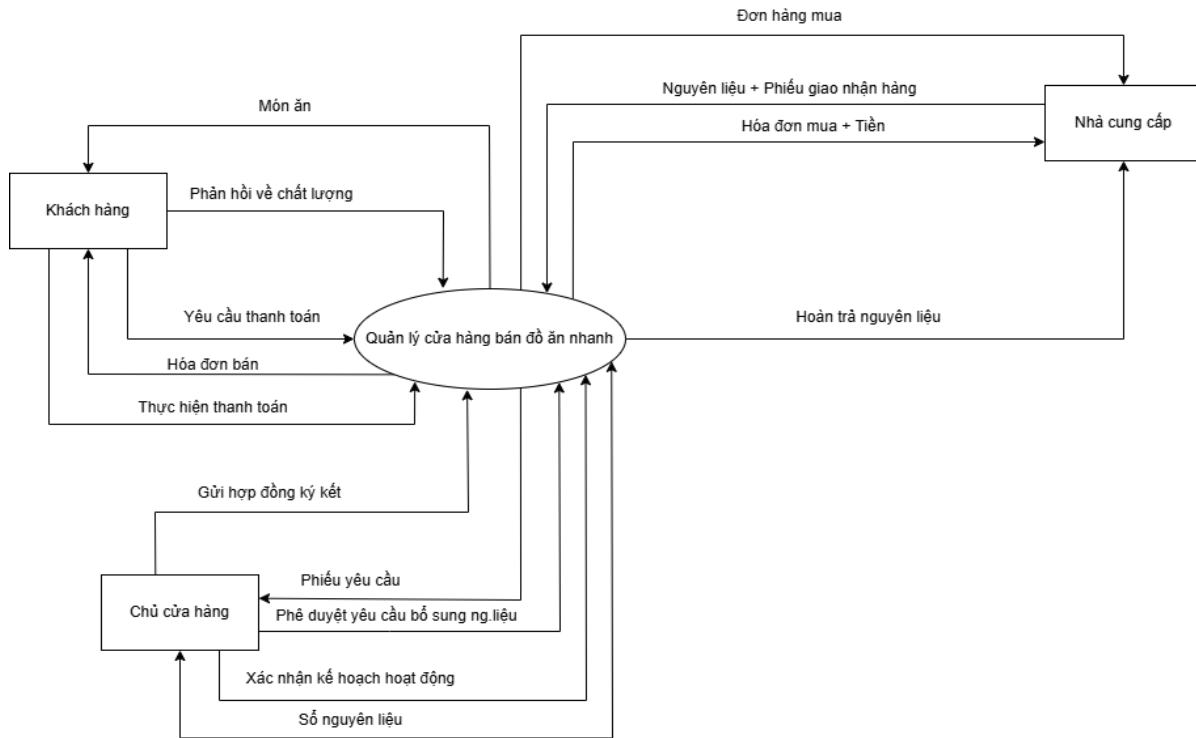
Việc sao lưu và phục hồi dữ liệu góp phần đảm bảo hệ thống hoạt động ổn định và hạn chế rủi ro mất mát dữ liệu trong quá trình vận hành.

4.7. Một số thiết kế chi tiết theo kiến trúc được chọn

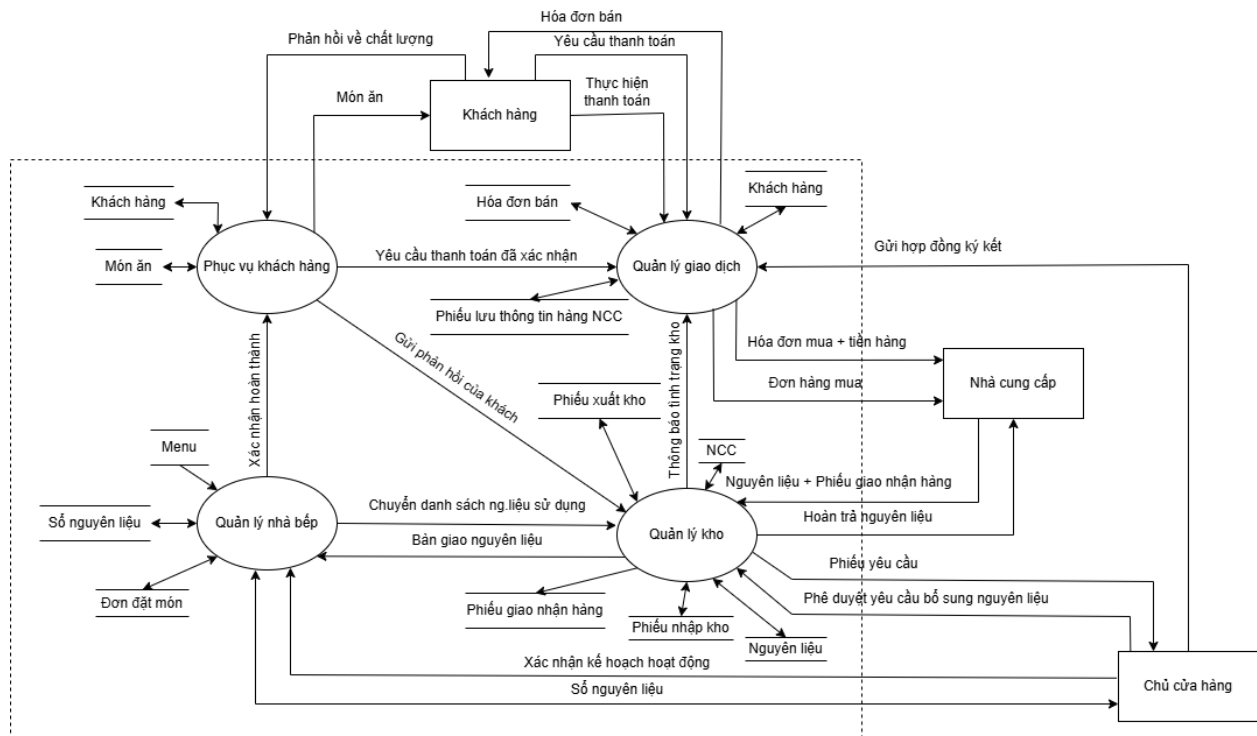
Sơ đồ phân rã chức năng



Sơ đồ DFD mức khung cảnh

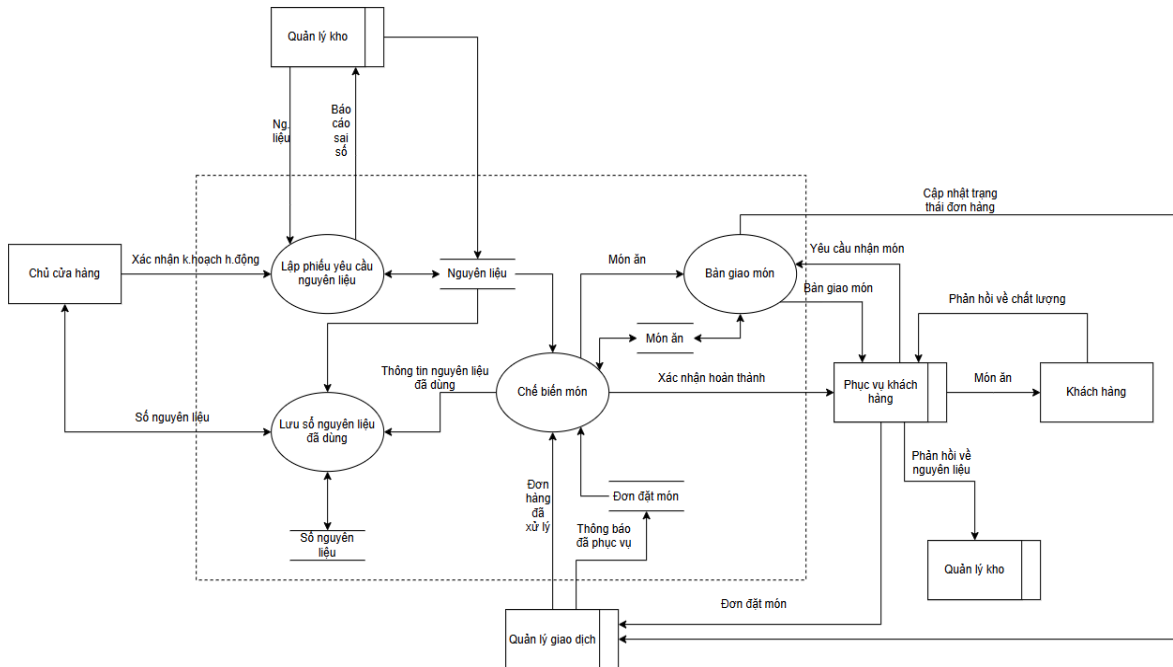


Sơ đồ DFD mức đỉnh

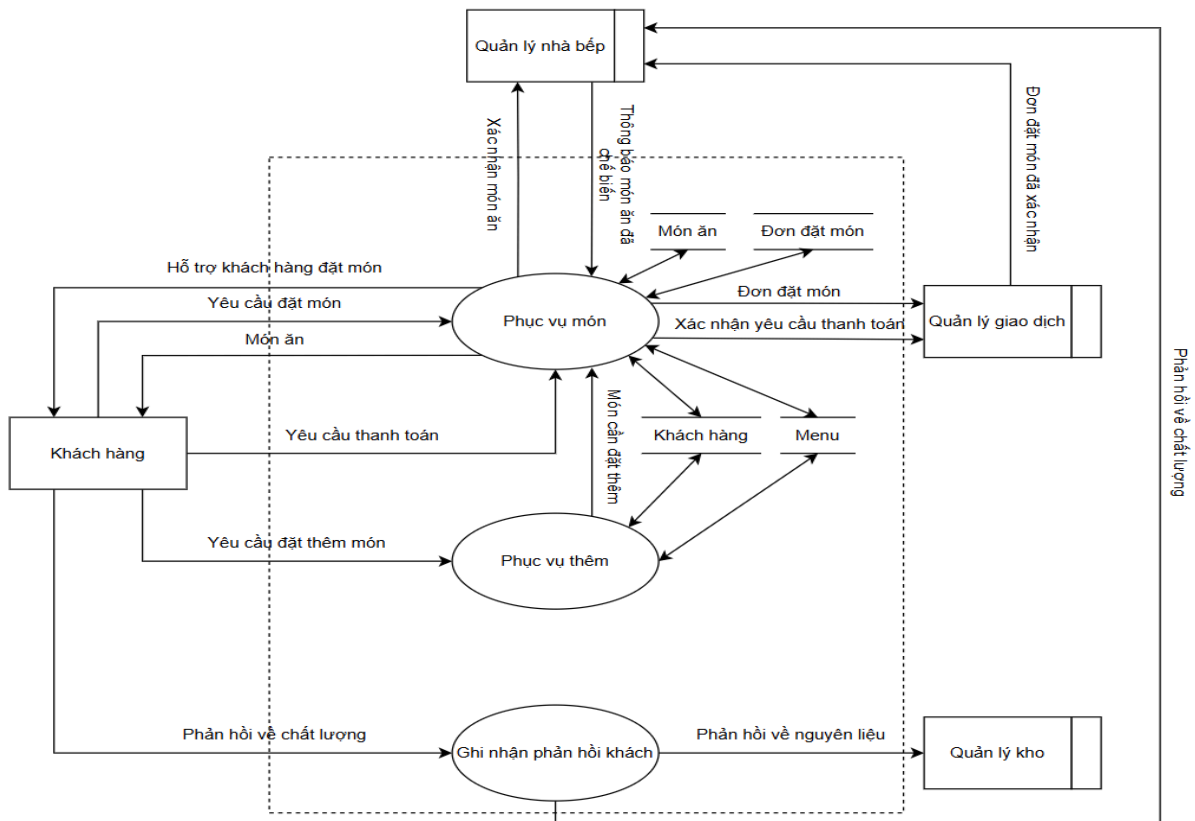


Sơ đồ DFD mức dưới đỉnh:

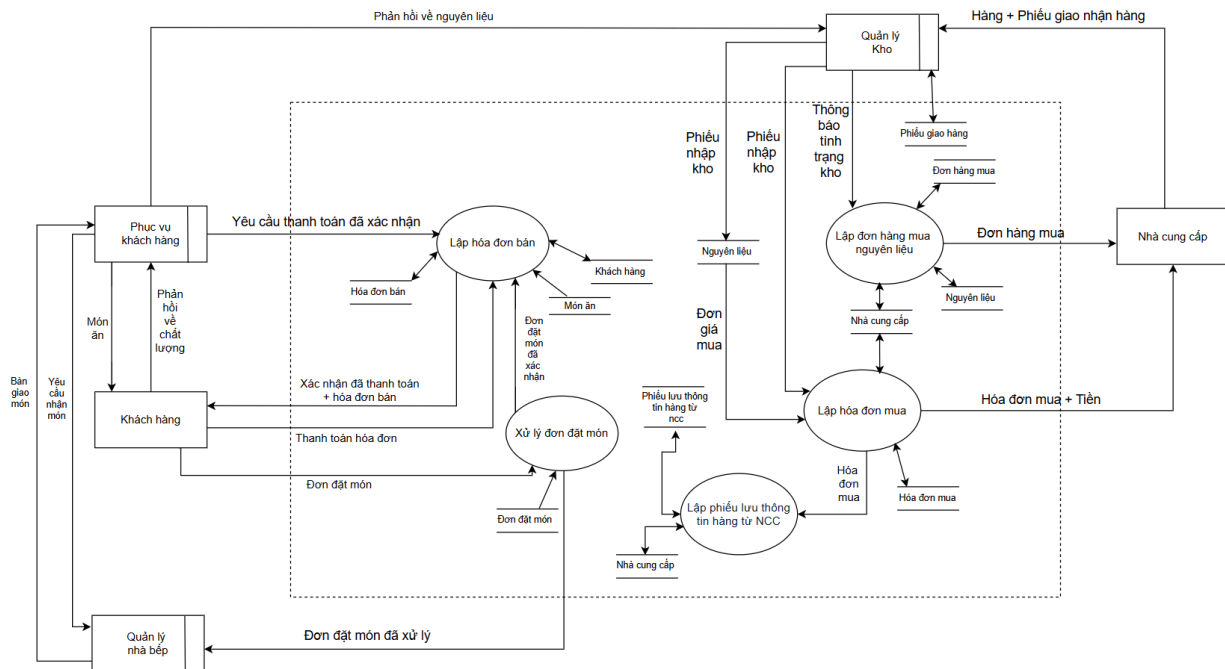
-Tiến trình quản lý nhà bếp



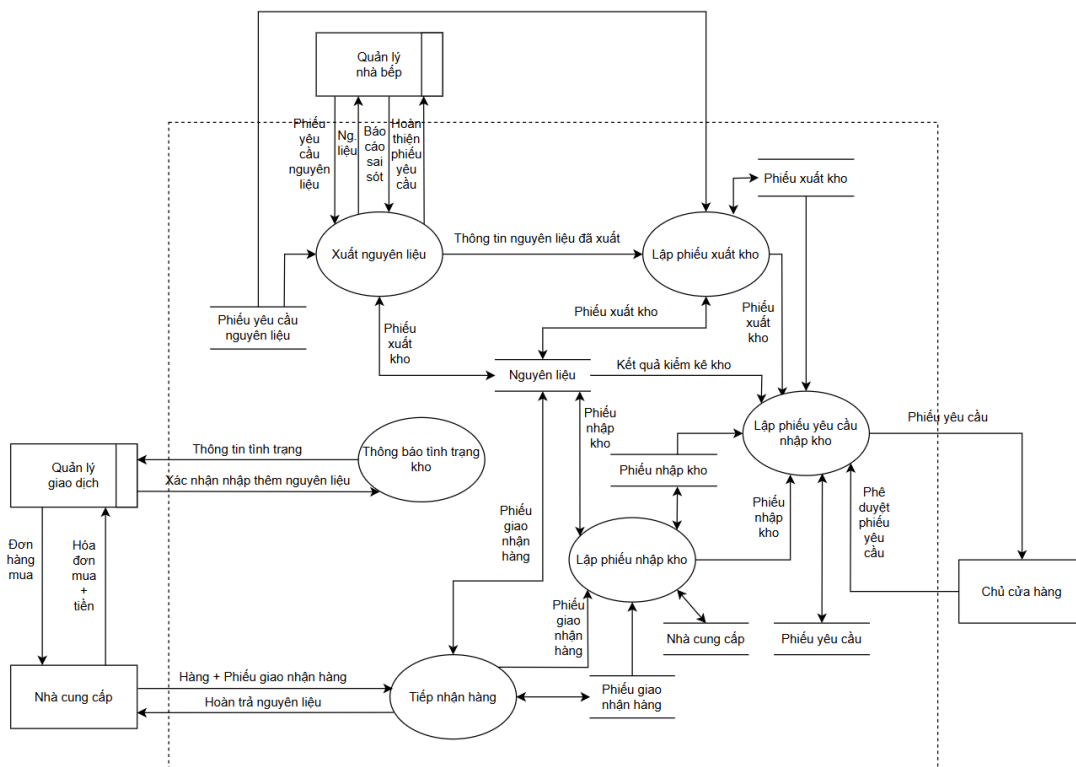
-Tiến trình phục vụ khách hàng



-Tiến trình quản lý giao dịch



-Tiến trình quản lý kho



CHƯƠNG 5. XÂY DỰNG VÀ KIỂM THỬ HỆ THỐNG

Hệ thống quản lý bán hàng FAF được xây dựng dựa trên kiến trúc phân lớp 3 tầng (Three-tier Architecture), tuân thủ mô hình Client-Server. Quá trình hiện thực hóa tập trung vào việc chuyển đổi các yêu cầu chức năng từ Product Backlog thành các module phần mềm hoạt động ổn định trên nền tảng Java Spring Boot và ReactJS.

5.1. Kết quả xây dựng module/nhóm chức năng

Dựa trên Product Backlog và các yêu cầu chức năng đã xác định trong tài liệu, nhóm đã lựa chọn xây dựng các module quan trọng nhất của hệ thống, tập trung vào các nghiệp vụ cốt lõi gồm: **đặt món, xử lý bếp, thanh toán và quản lý kho**.

Các chức năng này tương ứng với khoảng **90% yêu cầu quan trọng nhất**, đảm bảo hệ thống có thể vận hành được theo luồng thực tế của cửa hàng đồ ăn nhanh.

5.1.1. Danh sách các module chính

1. Module Quản lý Hệ thống & Phân quyền

- **Chức năng chính:** Quản lý thông tin nhân viên; thực hiện xác thực đăng nhập và phân quyền thao tác cho các bộ phận (Admin, Thu ngân, Bếp).
- **Dữ liệu liên quan:** Tài khoản nhân viên, nhóm quyền.
- **Công nghệ sử dụng:** Java Spring Boot, ReactJS (Frontend), Axios (Gọi API).

2. Module Quản lý Thực đơn & Định mức (Catalog & Recipe)

- **Chức năng chính:** Quản lý danh mục món ăn (Food, Drink, Combo); thiết lập công thức định lượng nguyên liệu cho từng món ăn để tự động tính toán tiêu hao.
- **Dữ liệu liên quan:** Danh mục món ăn, món ăn, định mức nguyên liệu.
- **Công nghệ sử dụng:** Java Spring Boot, JPA/Hibernate (ORM truy xuất dữ liệu), MySQL, ReactJS, Ant Design (Giao diện UI).

3. Module Bán hàng & Thu ngân (Sales)

- **Chức năng chính:** Hiển thị thực đơn trực quan; hỗ trợ thu ngân chọn bàn, lên đơn hàng, xử lý thanh toán và in hóa đơn cho khách.
- **Dữ liệu liên quan:** Đơn hàng, chi tiết đơn hàng, thông tin bàn.
- **Công nghệ sử dụng:** Java Spring Boot, ReactJS, Ant Design, WebSocket (STOMP/SockJS) để đồng bộ trạng thái bàn.

4. Module Quản lý Bếp (Kitchen)

- **Chức năng chính:** Tiếp nhận danh sách món cần chế biến từ thu ngân; cho phép nhân viên bếp cập nhật trạng thái món (Đang nấu, Đã xong) để bộ phận phục vụ trả món.
- **Dữ liệu liên quan:** Danh sách món chờ chế biến, chi tiết món theo đơn hàng.
- **Công nghệ sử dụng:** : Java Spring Boot, ReactJS, WebSocket (STOMP/SockJS) để nhận thông báo đơn ngay lập tức.

5. Module Quản lý Kho (Inventory)

- **Chức năng chính:** Theo dõi số lượng tồn kho nguyên liệu thực tế; lập phiếu nhập kho và tự động trừ tồn kho dựa trên hóa đơn bán hàng và định mức.
- **Dữ liệu liên quan:** Nguyên liệu, phiếu nhập kho.
- **Công nghệ sử dụng:** Java Spring Boot , JPA/Hibernate, ReactJS, Axios.

6. Module Báo cáo & Thống kê (Dashboard)

- **Chức năng chính:** Tổng hợp doanh thu theo thời gian; thống kê món ăn bán chạy; tính toán giá vốn và lợi nhuận cửa hàng.
- **Dữ liệu liên quan:** Hóa đơn bán hàng, dữ liệu tiêu hao nguyên liệu.
- **Công nghệ sử dụng:** Java Spring Boot, ReactJS, Chart.js.

5.1.2. Kết quả triển khai

- **Frontend:** Sử dụng ReactJS để hiển thị giao diện người dùng thân thiện và trực quan.
- **Backend:** Dùng Spring Boot để xử lý tập trung các logic nghiệp vụ của hệ thống.

- **Database:** MySQL được thiết kế tối ưu hóa với cấu trúc các bảng quan hệ chuẩn.

5.2. Kiểm thử hệ thống

5.2.1. Tổng quan về kiểm thử phần mềm

Kiểm thử phần mềm là một tiến trình trong vòng đời phát triển phần mềm (SDLC) nhằm đánh giá chất lượng sản phẩm, đảm bảo hệ thống hoạt động đúng theo các yêu cầu đã đặc tả.

- **5.2.1.1. Mục tiêu của kiểm thử:** * Phát hiện các lỗi logic, lỗi bảo mật và lỗi giao diện trước khi bàn giao cho cửa hàng FAF.
 - Đảm bảo tính chính xác trong việc tính toán hóa đơn và trừ kho nguyên liệu.
 - Xây dựng sự tin tưởng vào chất lượng sản phẩm cho người vận hành (Thu ngân, Bếp, Admin).
- **5.2.1.2. Các mức độ kiểm thử áp dụng:** * **Kiểm thử đơn vị (Unit Testing):** Kiểm tra các hàm xử lý nhỏ nhất trong Backend (ví dụ: hàm tính tổng tiền, hàm cập nhật stock).
 - **Kiểm thử tích hợp (Integration Testing):** Kiểm tra sự kết nối giữa Frontend (ReactJS) và Backend (API Spring Boot).
 - **Kiểm thử hệ thống (System Testing):** Kiểm tra toàn bộ luồng nghiệp vụ từ lúc gọi món đến khi xuất báo cáo.
 - **Kiểm thử nghiệm thu (Acceptance Testing):** Đánh giá mức độ hài lòng của người dùng cuối đối với giao diện và tính năng.

5.2.2. Mô hình kiểm thử được áp dụng trong đề tài (Agile Testing)

Do dự án được quản lý theo phương pháp **Agile Scrum**, nhóm không áp dụng các mô hình kiểm thử truyền thống (như Thác nước hay chữ V) mà áp dụng phương pháp **Kiểm thử linh hoạt (Agile Testing)**. Trong mô hình này, hoạt động kiểm thử diễn ra liên tục và song song với quá trình phát triển phần mềm trong mỗi Sprint:

- **Tính liên tục:** Ngay khi một User Story (ví dụ: "Tạo đơn hàng") được Dev hoàn thành mã nguồn (**SalesController.java**), Tester sẽ tiến hành kiểm thử ngay lập tức thay vì đợi xong toàn bộ hệ thống.
- **Tiêu chí chấp nhận (Acceptance Criteria):** Các kịch bản kiểm thử (Test Case) được xây dựng trực tiếp từ các tiêu chí chấp nhận của User Story trong Product Backlog.

- **Phản hồi nhanh:** Lỗi được phát hiện và báo cáo ngay trong các buổi họp Daily Scrum để Dev khắc phục kịp thời ngay trong Sprint hiện tại.

5.2.3. Kế hoạch và thực thi kiểm thử theo quy trình Agile

Kế hoạch kiểm thử hệ thống "Quản lý cửa hàng đồ ăn nhanh FAF" được Nhóm 5 chia nhỏ theo từng Sprint và thực hiện qua quy trình 9 bước lặp lại (Iterative Process), bám sát vào kiến trúc hệ thống (Spring Boot & ReactJS):

Bước 1: Phân tích yêu cầu và xác định mục tiêu kiểm thử (Sprint Planning)

Ngay từ buổi họp đầu mỗi Sprint, nhóm tiến hành phân tích các User Story để xác định "Định nghĩa hoàn thành" (Definition of Done - DoD).

- **Mục tiêu:** Xác định các điểm chạm (touchpoints) cần kiểm tra.
- **Đầu ra:** Danh sách các kịch bản kiểm thử biên (Edge cases) cho từng chức năng mới.

Bước 2: Lập kế hoạch và phân bổ nguồn lực (Resource Allocation)

Nhóm phân chia vai trò rõ ràng để tối ưu hóa thời gian:

- **Developer (Lập trình viên):** Chịu trách nhiệm viết Unit Test cho các lớp Logic tại Backend để đảm bảo tính đúng đắn của hàm tính toán trước khi đẩy code.
- **Tester (Kiểm thử viên - Thành viên nhóm):** Thực hiện kiểm thử chức năng (Functional Testing) và kiểm thử giao diện (UI Testing) trên các thiết bị Tablet và PC.
- **Công cụ hỗ trợ:** Postman (kiểm tra API), JUnit 5 (kiểm tra Logic), Các loại DevTools của các trình duyệt như Chrome, Microsoft Edge, ... (kiểm tra Response Time và lỗi CSS).

Bước 3: Thiết kế kịch bản kiểm thử hệ thống (Test Case Design)

Dựa trên kiến trúc Client-Server kết hợp Three-Tier và 5 Sprint, nhóm xây dựng bộ kịch bản kiểm thử tập trung vào tính liên kết:

- **Kiểm thử giao diện:** Xác thực các Component React (như Menu, Giỏ hàng) hiển thị đúng hình ảnh từ thư mục `/images`.

- **Kiểm thử nghiệp vụ:** Theo dõi luồng dữ liệu từ **apiClient.js** gửi đến các Controller tương ứng.

Test Case :

Mã TC	Chức năng kiểm thử	Mục tiêu	Điều kiện tiên quyết	Dữ liệu kiểm thử	Độ quan trọng	Kết quả mong đợi	Kết quả thực tế	Đánh giá
TC01	Đăng nhập Admin	Kiểm tra đăng nhập và giao diện Admin	Có tài khoản Admin	ADMIN/ đúng pass	Cao	Đăng nhập thành công, hiển thị giao diện Admin	Hệ thống chuyển hướng đúng trang Admin, hiển thị đầy đủ menu quản trị.	PASS
TC02	Đăng nhập Thu ngân	Kiểm tra giao diện đúng vai trò Thu ngân	Có tài khoản Thu ngân	ThuNgan / đúng pass	Cao	Hiển thị đúng màn hình thanh toán/đơn hàng	Chuyển hướng về trang Thu Ngân , hiển thị danh sách bàn và menu gọi món.	PASS
TC03	Đăng nhập Bếp	Kiểm tra giao diện đúng	Có tài khoản Bếp	Bep/ đúng pass	Cao	Hiển thị danh sách	Chuyển hướng về Bếp , danh sách	PASS

		vai trò Bếp				đơn cho bếp	đơn hàng được load realtime.	
TC04	Đăng nhập sai	Kiểm tra xử lý sai mật khẩu	Có tài khoản hợp lệ	ADMIN/ sai pass	Cao	Hiện thị thông báo lỗi	Popup thông báo "Sai tài khoản hoặc mật khẩu" xuất hiện đúng lúc.	PASS
TC05	Xem menu	Kiểm tra hiển thị món ăn	Có dữ liệu món	Danh sách foods	Trung bình	Hiện thị đúng tên, giá, danh mục	Hình ảnh và thông tin món ăn load mượt từ Database.	PASS
TC06	Tạo đơn hàng	Kiểm tra tạo đơn	Có dữ liệu món	Burger x2	Cao	Tạo đơn thành công, lưu DB	Record mới được tạo trong bảng orders, UI cập nhật trạng thái "Đang chờ".	PASS
TC07	Thêm món	Kiểm tra	Đã có đơn	Khoai tây x1	Trung bình	Đơn cập	Đơn hàng cập nhật	PASS

		thêm món vào đơn				nhật đúng, tổng tiền thay đổi	số lượng món mới, giá trị đơn hàng nhảy số ngay lập tức.	
TC08	Gửi đơn xuống bếp	Kiểm tra luồng đơn	Đã tạo đơn	Đơn bàn 5	Cao	Hiện thị đơn ở bếp	WebSockets gửi signal thành công, màn hình Bếp hiện đơn mới sau 1s.	PASS
TC09	Cập nhật trạng thái món	Kiểm tra xử lý trạng thái	Có đơn ở bếp	Đơn bàn 5	Cao	Trạng thái cập nhật thành công	Trạng thái chuyển từ Pending sang Processing khi Bếp click xác nhận.	PASS
TC10	Lập hóa đơn	Kiểm tra tạo hóa đơn	Có đơn hàng	Bàn 5	Cao	Tạo hóa đơn thành công	Bản ghi trong bảng Invoices được tạo, ID hóa đơn khớp	PASS

							với ID đơn hàng.	
TC11	Tính tổng tiền	Kiểm tra tính toán	Đơn có nhiều món	2 Burger + 1 Pepsi	Cao	Tổng tiền chính xác	Logic Backend tính toán đúng (Số lượng x Đơn giá), không sai lệch dù đơn dài.	PASS
TC12	Thanh toán	Kiểm tra hoàn tất thanh toán	Có hóa đơn	Tiền mặt	Cao	Thanh toán thành công, lưu DB	Thanh toán xong sẽ gửi thông tin đơn xuống bếp, sau khi bếp phục vụ và khách ăn xong sẽ được nhân viên thu ngân xác thực và giải phóng bàn.	PASS
TC13	Nhập kho	Kiểm tra tạo	Có dữ liệu NCC	Thịt bò 10kg	Trung bình	Tạo phiếu nhập	Thông tin nhà cung cấp và số	PASS

		phiếu nhập				thành công	lượng nguyên liệu được lưu vào stock_rec eipts.	
TC14	Cập nhật tồn kho	Kiểm tra số lượng tồn	Đã nhập kho	+10kg	Trung bình	Tồn kho tăng đúng	Số lượng trong bảng ingredients tăng đúng bằng lượng nhập thực tế.	PASS

Bước 4: Thiết lập môi trường kiểm thử (Environment Setup)

Môi trường Test được tách biệt hoàn toàn để không ảnh hưởng đến dữ liệu phát triển:

- **Dữ liệu mẫu:** Sử dụng script

db/backfill/backfill_ingredient_import_price.sql để thiết lập trạng thái kho và thực đơn tiêu chuẩn.

- **Cấu hình:** Đảm bảo Backend (Port 8080/8081) và Frontend (Port 5173) có thể giao tiếp qua mạng LAN/Intranet để mô phỏng điều kiện thực tế của cửa hàng.

Bước 5: Thực thi kiểm thử trong Sprint (Execution)

Đây là bước quan trọng nhất, được thực hiện lặp đi lặp lại:

- **Kiểm thử API:** Sử dụng Postman để kiểm tra các phương thức GET/POST/PUT/DELETE. Đảm bảo mã phản hồi luôn là 200 OK cho các yêu cầu hợp lệ.
- **Kiểm thử Real-time:** Đặc biệt quan trọng trong Sprint 2. Nhóm thực hiện mở song song hai màn hình (Thu ngân và Bếp). Khi thực hiện đặt món trên Tablet, nhóm quan sát sự thay đổi trạng thái ngay lập tức trên màn hình Bếp **mà không cần tải lại trang.**

Bước 6: Báo cáo và quản lý lỗi (Bug Tracking)

Các lỗi phát hiện được thảo luận trực tiếp trong buổi **Daily Scrum** (15 phút + mỗi ngày):

- Lỗi được phân loại theo mức độ: *Critical (Nghiêm trọng - gây sập hệ thống)*, *Major/Cao (Lỗi chức năng)*, *Minor/Trung bình (Lỗi giao diện)*.
- Nhóm thảo luận và sửa lỗi trực tiếp trong buổi họp Daily Scrum

Bước 7: Kiểm thử hồi quy (Regression Testing)

Sau khi hoàn tất hết các khâu code (đã xác định chạy được, hoạt động được của từng sprint), nhóm mới mang bộ 15 Test Case ra để chạy từ đầu đến cuối một lượt. Mục đích là để đảm bảo việc ghép code của Sprint 5 (Kho) không vô tình làm hỏng logic code của Sprint 1 (Đặt món). Ghi nhận kết quả "Kết quả thực tế" và đánh giá PASS/FAIL vào bảng.

Bước 8: Kiểm thử nghiệm thu (Sprint Review & UAT)

Cuối mỗi Sprint, nhóm tổ chức buổi Demo cho "khách hàng giả định":

- Thực hiện kịch bản "Một ngày tại cửa hàng FAF": Từ lúc mở bàn, khách gọi món, bếp chế biến, bồi bàn phục vụ đến khi thu ngân in hóa đơn và kiểm kho cuối ngày.
- Thu thập phản hồi để tinh chỉnh giao diện (UI) cho phù hợp với thao tác của nhân viên.

Bước 9: Đóng gói và bàn giao hồ sơ kiểm thử

Sau Sprint 5, nhóm tổng kết toàn bộ quá trình:

- Xuất file báo cáo kết quả kiểm thử.
- Lưu trữ các bộ script test và Postman Collection vào Repository của dự án.
- Xác nhận hệ thống đạt trạng thái "Release Ready" (Sẵn sàng bàn giao).

5.2.4. Kết quả kiểm thử

Sau khi hoàn thành giai đoạn thực thi kiểm thử hệ thống (System Testing) dựa trên bộ 15 kịch bản đã thiết kế, nhóm 5 tổng hợp kết quả như sau:

5.2.4.1. Thống kê tổng quát

Quá trình kiểm thử ghi nhận sự ổn định cao của hệ thống sau khi đã trải qua các đợt Fix Bug liên tục trong từng Sprint.

- **Tổng số kịch bản kiểm thử (Test Case):** 15
- **Số kịch bản Đạt (Pass):** 15
- **Số kịch bản Không đạt (Fail):** 0
- **Tỷ lệ thành công:** 100%
- **Tổng số lỗi (Bugs) phát hiện trong toàn bộ quá trình phát triển:** 12 lỗi (Đã khắc phục 100%).

5.2.4.2. Phân tích kết quả theo các phân hệ nghiệp vụ

Kết quả được đánh giá dựa trên sự phối hợp giữa Backend (Spring Boot) và Frontend (ReactJS):

- **Phân hệ Bán hàng & Bếp (Sprint 1 & 2):** Các kịch bản từ TC02 đến TC07 cho thấy luồng dữ liệu từ Tablet của khách hàng được đẩy xuống Bếp với độ trễ cực thấp (< 1s). Công nghệ WebSocket hoạt động ổn định, đảm bảo màn hình Bếp cập nhật trạng thái món ăn thời gian thực mà không cần F5 trình duyệt.
- **Phân hệ Thanh toán & Hóa đơn (Sprint 3):** Các kịch bản TC08 đến TC10 xác nhận logic tính toán tại `SalesServiceImpl` chính xác tuyệt đối. Hệ

thống xử lý tốt việc gộp đơn và chuyển trạng thái bàn từ "Có khách" sang "Trống" ngay sau khi in hóa đơn thành công.

- **Phân hệ Kho & Quản trị (Sprint 4 & 5):** Các kịch bản từ TC11 đến TC15 chứng minh tính năng trừ kho tự động hoạt động đúng định mức đã thiết lập. Dữ liệu trên Dashboard khớp hoàn toàn với lịch sử hóa đơn trong Database MySQL.

5.2.4.3. Danh sách các lỗi tiêu biểu và tình trạng khắc phục

Dưới đây là một số lỗi nghiêm trọng (Major Bugs) đã được phát hiện trong quá trình "Test & Fix" cuốn chiếu và đã được xử lý triệt để trước khi đóng gói:

- Lỗi về API : Không liên kết được Frontend tới Backend

5.2.4.4. Đánh giá chất lượng hệ thống (Final Quality Assessment)

Thông qua kết quả kiểm thử thực tế, nhóm 5 đưa ra các đánh giá sau:

- **Tính ổn định:** Hệ thống vận hành trơn tru, không xảy ra hiện tượng treo hoặc sập Server khi thực hiện nhiều giao dịch cùng lúc.
- **Tính chính xác:** Toàn bộ dữ liệu về tài chính và tồn kho được xử lý chính xác theo đúng đặc tả yêu cầu nghiệp vụ.
- **Trải nghiệm người dùng:** Giao diện trên Tablet mượt mà, các thông báo thời gian thực giúp giảm thiểu sai sót và tăng tốc độ phục vụ tại cửa hàng.

Checklist:

STT	Hạng mục rà soát	Trạng thái	Nhận xét ngắn
1	Repo đã tách Frontend và Backend	Đạt	Có 2 thư mục frontend và backend rõ ràng.
2	Frontend đã khởi tạo project React	Đạt	Đã cấu hình đầy đủ React + Vite.
3	Frontend đã render ứng dụng từ main.jsx	Đạt	Ứng dụng được mount và chạy ổn định.
4	Frontend đã có giao diện nghiệp vụ POS/Shop	Đạt	Đã xây dựng giao diện POS phục vụ bán hàng.
5	Frontend đã chia component theo module	Đạt	Các component được tách theo từng chức năng.
6	Frontend đã cài Ant Design	Đạt	Đã tích hợp Ant Design cho giao diện người dùng.
7	Frontend đã cài Axios để gọi API	Đạt	Đã sử dụng Axios để kết nối backend.
8	Frontend đã dùng React Router	Đạt	Đã sử dụng React Router để quản lý điều hướng.
9	Backend đã khởi tạo project Spring Boot	Đạt	Đã thiết lập project Spring Boot hoàn chỉnh.
10	Backend đã có class main chạy ứng dụng	Đạt	Ứng dụng backend chạy ổn định.

11	Backend đã cấu hình tên ứng dụng	Đạt	Đã cấu hình trong application.properties.
12	Backend đã khai báo dependency Web	Đạt	Đã sử dụng Spring Web MVC.
13	Backend đã khai báo dependency JPA	Đạt	Đã tích hợp Spring Data JPA.
14	Backend đã khai báo MySQL Connector	Đạt	Đã kết nối với MySQL thông qua driver.
15	Backend đã có Controller API	Đạt	Đã xây dựng các API phục vụ hệ thống.
16	Backend đã có Entity/Repository/Service	Đạt	Đã triển khai đầy đủ các lớp nghiệp vụ.
17	Backend đã cấu hình kết nối DB chi tiết	Đạt	Đã cấu hình kết nối database và JPA.
18	Backend đã có test cơ bản	Đạt	Đã có test kiểm tra khởi động hệ thống.
19	Backend đã có test nghiệp vụ/API	Đạt	Đã xây dựng test cho các chức năng chính.
20	FE-BE đã kết nối với nhau	Đạt	Frontend và backend đã giao tiếp qua REST API.

CHƯƠNG 6. KẾT LUẬN

6.1. Đánh giá chất lượng phần mềm

Dựa trên kết quả triển khai và kiểm thử, hệ thống "Quản lý cửa hàng đồ ăn nhanh FAF" được đánh giá đáp ứng tốt các tiêu chuẩn đề ra:

- Về tính năng: Hoàn thiện luồng nghiệp vụ khép kín từ khâu khách đặt món (Tablet), Bếp tiếp nhận, Thu ngân thanh toán cho đến quản trị Tài kho tự động.
- Về hiệu năng & Trải nghiệm: Việc sử dụng kiến trúc SPA (ReactJS) kết hợp Spring Boot giúp hệ thống phản hồi nhanh, giao diện mượt mà không cần tải lại trang.
- Về tính ổn định: Công nghệ WebSocket (SockJS/STOMP) hoạt động hiệu quả, đảm bảo thông báo thời gian thực giữa các thiết bị với độ trễ cực thấp ($< 1s$).

6.2. Ưu điểm & Nhược điểm của hệ thống

6.2.1. Ưu điểm

- Áp dụng công nghệ hiện đại: Sử dụng bộ stack phổ biến (Spring Boot, React/Vite, SQL Server) giúp phần mềm có tính ứng dụng thực tiễn cao.
- Giao tiếp thời gian thực (Real-time): Giải quyết triệt để bài toán ùn tắc thông tin giữa bộ phận Bán hàng và nhà Bếp trong giờ cao điểm.
- Tự động hóa quản lý kho: Hệ thống tự động đối chiếu và trừ hao hụt nguyên liệu theo định mức ngay khi đơn hàng hoàn tất.
- Quy trình Agile hiệu quả: Việc chia nhỏ dự án thành 5 Sprint giúp nhóm kiểm soát mã nguồn và xử lý lỗi (Bug) cực kỳ sát sao.

6.2.2. Nhược điểm (Hạn chế)

- Phụ thuộc mạng nội bộ: Luồng công việc Bếp – Thu ngân dựa hoàn toàn vào mạng LAN/Wi-Fi, nếu rớt mạng hệ thống sẽ bị gián đoạn.
- Thanh toán mô phỏng: Tính năng mã QR/Ví điện tử mới dừng ở mức mô phỏng logic, chưa kết nối trực tiếp với API ngân hàng thật.

- Chưa tối ưu bộ nhớ đệm (Cache): Mỗi thao tác load Menu đều phải truy vấn trực tiếp xuống Database, có thể gây quá tải nếu lượng thiết bị kết nối quá lớn.
- Giới hạn mô hình: Kiến trúc cơ sở dữ liệu hiện tại chỉ tối ưu cho một cửa hàng đơn lẻ (Single Store), chưa hỗ trợ quản lý chuỗi chi nhánh.

6.3. Hướng phát triển

Để hoàn thiện và có thể đưa phần mềm vào ứng dụng thương mại, nhóm đề xuất các hướng nâng cấp sau:

1. Tích hợp thanh toán thật: Kết nối với các API Sandbox của VNPay, MoMo để xử lý giao dịch điện tử trọn vẹn.
2. Tối ưu hệ thống (Scaling): Tích hợp Redis Cache để lưu đệm dữ liệu Menu/Trạng thái bàn. Đóng gói ứng dụng bằng Docker để dễ dàng triển khai lên Cloud.
3. Mở rộng mô hình chuỗi (Multi-tenant): Nâng cấp Database để hỗ trợ quản lý nhiều chi nhánh, phân cấp kho tổng và kho lẻ.
4. Phát triển hệ sinh thái: Xây dựng thêm Web/Mobile App cho khách hàng đặt đồ ăn từ xa (Take-away) và tích hợp hệ thống thẻ thành viên (Loyalty Program).

TÀI LIỆU THAM KHẢO

Tài liệu chính:

[79_9_NguyenHuuTin_QLBH_doannhanh.docx](#)

[Tài liệu của thầy Thành](#)

[Tài liệu khóa trước](#)

[Tài liệu môn QLDA](#)

PHỤ LỤC A: QUY TẮC ĐẶT TÊN

Tên file tài liệu cần được đặt theo một quy tắc thống nhất nhằm đảm bảo dễ quản lý, truy xuất và phân loại.

Cấu trúc đặt tên file tài liệu của hệ thống như sau:

2.2.1 Tên file tài liệu:

- Cấu trúc: [Loại tài liệu][Tên dự án][Mô tả ngắn]_[Ngày tạo].[Định dạng]
- Trong đó:

[Loại tài liệu]: Viết tắt loại tài liệu

[Tên dự án]: Tên dự án hoặc mã dự án viết ngắn gọn.

[Mô tả ngắn]: Mô tả nội dung tài liệu để dễ phân biệt.

[Ngày tạo]: Định dạng YYYYMMDD (Năm-Tháng-Ngày) để dễ sắp xếp theo thời gian.

[Định dạng]: Phần mở rộng của file, ví dụ: .docx, .xlsx, .pdf.

- Ví dụ: SRS_ECOM_UserModule_20260317.pdf

(SRS=Software Requirements Specification/ ECOM=Ecommerce Project)

2.2.2. Tên biến trong mã nguồn:

Quy tắc: Để đảm bảo mã nguồn dễ đọc, dễ bảo trì và tuân theo các chuẩn lập trình, nhóm sẽ tuân theo quy tắc đặt tên biến sau:

1. Sử dụng tiếng Anh: Tránh đặt tên biến bằng tiếng Việt có dấu hoặc không dấu.

2. Sử dụng kiểu CamelCase hoặc snake_case tùy theo loại biến:

- Biến cục bộ, tham số hàm: dùng camelCase, bắt đầu bằng chữ thường.
- Biến hằng số: dùng UPPER_CASE_SNAKE_CASE, viết hoa toàn bộ và phân cách bằng dấu gạch dưới _.
- Tên lớp, đối tượng: dùng PascalCase, mỗi chữ cái đầu của từ viết hoa.

3. Tên biến phải có ý nghĩa, tránh viết tắt khó hiểu:

- customerName, orderTotal, isValidUser
- cusNm, ordTtl, ivU

4. Tránh sử dụng các tên biến chung chung hoặc khó hiểu:

- productList (danh sách sản phẩm), userAge (tuổi của người dùng)
- data, temp, value

5. Tuân theo quy tắc đặt tên theo ngữ cảnh sử dụng:

2.2.3. Tên cơ sở dữ liệu:

- Quy tắc:

Tên CSDL phải ngắn gọn, dễ hiểu, phản ánh đúng mục đích của hệ thống.

Sử dụng tiếng Anh, không chứa khoảng trắng, có thể dùng dấu gạch dưới _ để phân tách từ.

Tránh sử dụng ký tự đặc biệt hoặc từ khóa hệ thống của SQL.

- Ví dụ:

warehouse_management (Hệ thống quản lý kho)

inventory_system (Hệ thống quản lý hàng tồn kho)

QLKhoHang (Dùng tiếng Việt không dấu, khó mở rộng)

2.2.4. Tên bảng trong cơ sở dữ liệu:

- Quy tắc:

Tên bảng phải phản ánh đúng nội dung dữ liệu được lưu trữ.

Sử dụng tiếng Anh, không chứa khoảng trắng, dùng dấu gạch dưới _ để phân tách từ.

Tên bảng ở dạng số nhiều để thể hiện chứa nhiều bản ghi.

- Ví dụ:

Đúng: users, order_details, products.

Sai: User, OrderDetails.

2.2.5. Tên khóa chính và khóa ngoại:

- Khóa chính (Primary Key – PK):

Tên khóa chính thường đặt theo dạng id_tên_bảng. Luôn là số nguyên (INT hoặc BIGINT), tự động tăng (AUTO_INCREMENT).

- Ví dụ:

id_user (Khóa chính của bảng users)

id_product (Khóa chính của bảng products)

id_order (Khóa chính của bảng orders)

- Khóa ngoại (Foreign Key – FK):

Tên khóa ngoại thường có dạng tên_bảng_được_tham_chiếu_id.

Đảm bảo tuân thủ ràng buộc FOREIGN KEY với bảng cha

PHỤ LỤC B: CÁC BIỂU MẪU, TÀI LIỆU THU THẬP ĐƯỢC

Link khảo sát:

[Khảo sát cửa hàng đồ ăn nhanh - Google Biểu mẫu](#)

Link kết quả khảo sát:

 Khảo sát cửa hàng đồ ăn nhanh (Câu trả lời)

PHỤ LỤC C: CÁC BIÊN BẢN TIẾN ĐỘ

Biên bản họp nhóm

Cuộc họp 1

- **Thời gian:** 31/03
- **Nội dung:**
 - Thảo luận đề tài
 - Phân chia công việc
- **Kết luận:**
 - Mỗi thành viên phụ trách một module (FE, BE, DB)

Cuộc họp 2

- **Thời gian:** 10/04
- **Nội dung:**
 - Kiểm tra tiến độ backend
 - Thảo luận API
- **Kết luận:**
 - Hoàn thiện API theo chuẩn REST

Cuộc họp 3

- **Thời gian:** 17/04
- **Nội dung:**
 - Review giao diện frontend
 - Thống nhất layout hệ thống
- **Kết luận:**
 - Sử dụng Ant Design cho UI

Cuộc họp 4

- **Thời gian:** 20/04
- **Nội dung:**
 - Test hệ thống
 - Chuẩn bị demo
- **Kết luận:**
 - Hoàn thiện các lỗi nhỏ và tối ưu UI

PHỤ LỤC D: CÁC BIÊN BẢN RÀ SOÁT KỸ THUẬT

Checklist:

STT	Hạng mục rà soát	Trạng thái	Nhận xét ngắn
1	Repo đã tách Frontend và Backend	Đạt	Có 2 thư mục frontend và backend rõ ràng.
2	Frontend đã khởi tạo project React	Đạt	Đã cấu hình đầy đủ React + Vite.
3	Frontend đã render ứng dụng từ main.jsx	Đạt	Ứng dụng được mount và chạy ổn định.
4	Frontend đã có giao diện nghiệp vụ POS/Shop	Đạt	Đã xây dựng giao diện POS phục vụ bán hàng.
5	Frontend đã chia component theo module	Đạt	Các component được tách theo từng chức năng.
6	Frontend đã cài Ant Design	Đạt	Đã tích hợp Ant Design cho giao diện người dùng.
7	Frontend đã cài Axios để gọi API	Đạt	Đã sử dụng Axios để kết nối backend.
8	Frontend đã dùng React Router	Đạt	Đã sử dụng React Router để quản lý điều hướng.
9	Backend đã khởi tạo project Spring Boot	Đạt	Đã thiết lập project Spring Boot hoàn chỉnh.
10	Backend đã có class main chạy ứng dụng	Đạt	Ứng dụng backend chạy ổn định.
11	Backend đã cấu hình tên ứng dụng	Đạt	Đã cấu hình trong application.properties.
12	Backend đã khai báo dependency Web	Đạt	Đã sử dụng Spring Web MVC.
13	Backend đã khai báo dependency JPA	Đạt	Đã tích hợp Spring Data JPA.

PHỤ LỤC E: HỒ SƠ TOÀN BỘ DỰ ÁN

1. Hồ sơ khảo sát

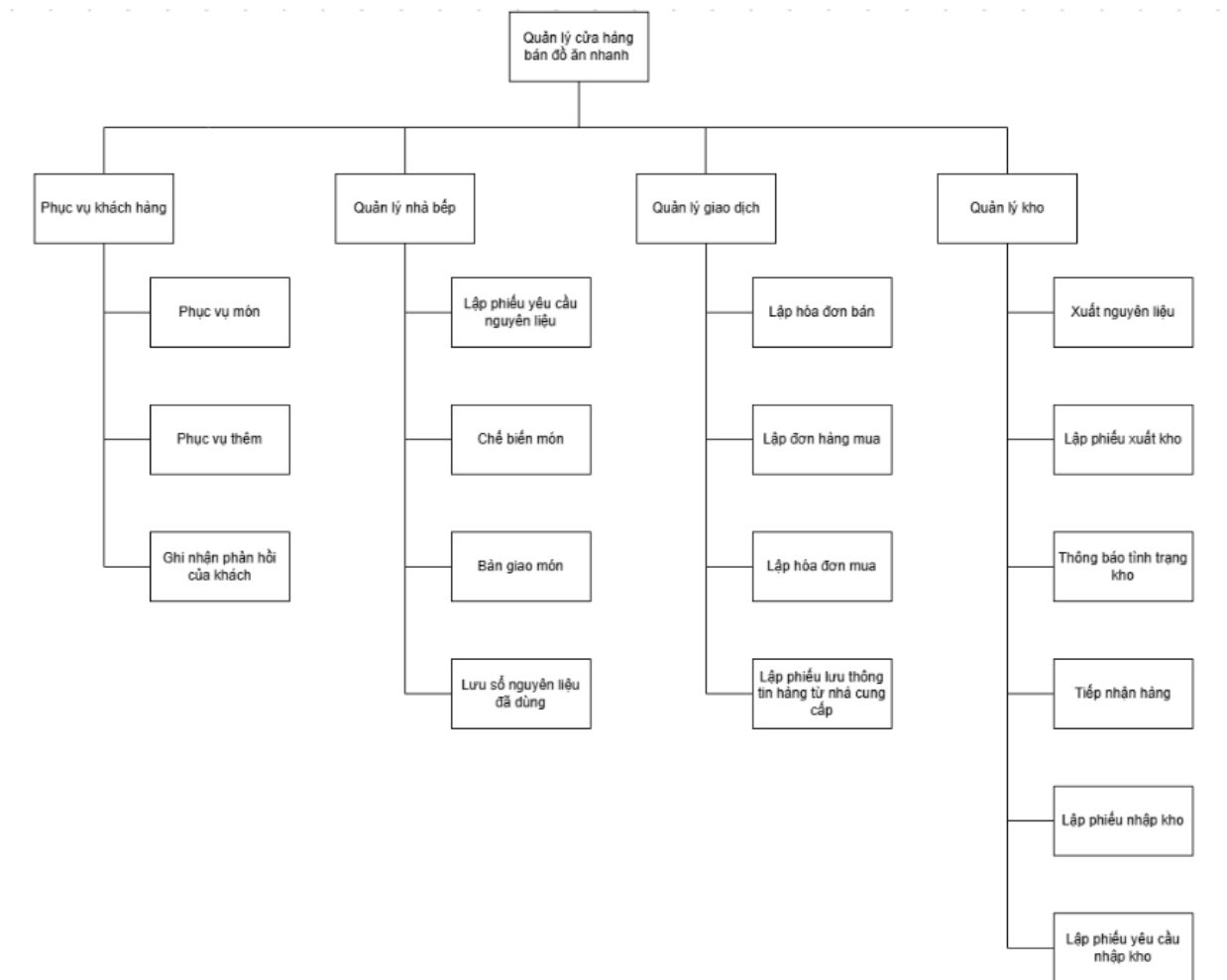
Link khảo sát:

[Khảo sát cửa hàng đồ ăn nhanh - Google Biểu mẫu](#)

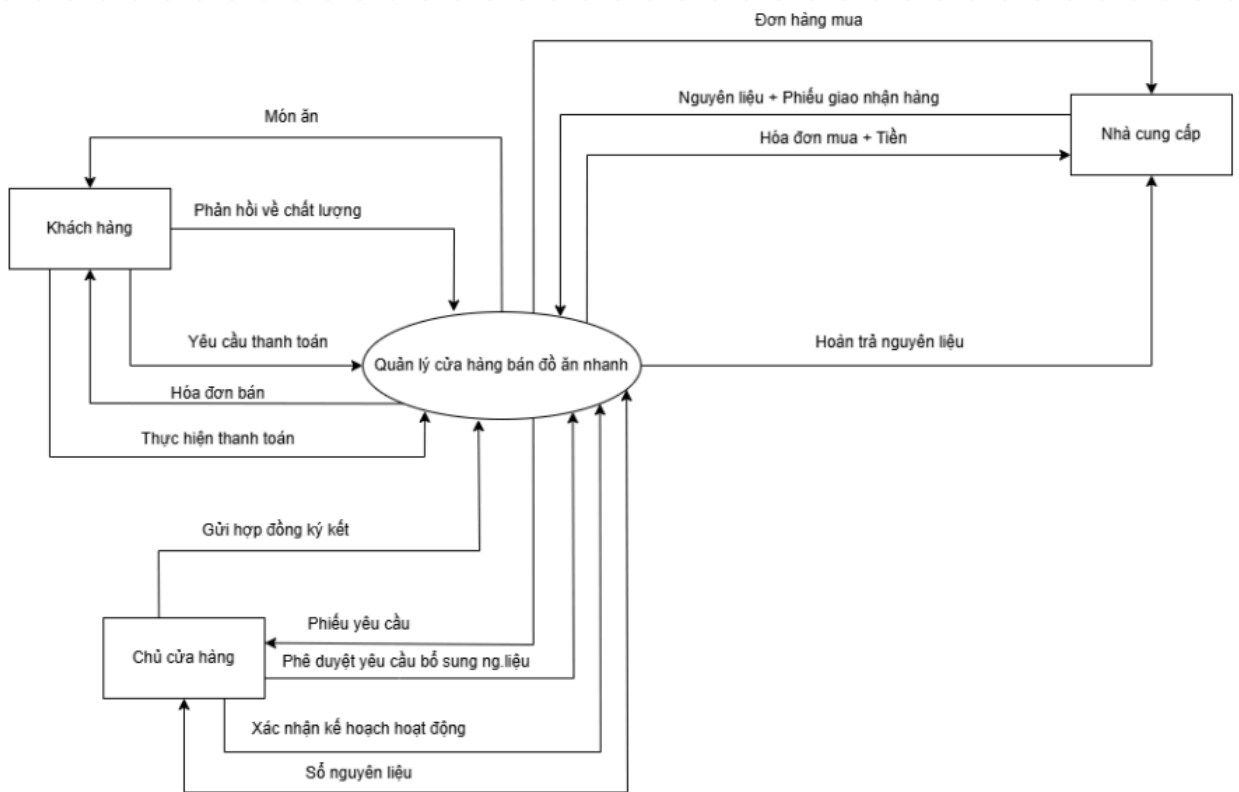
Link kết quả khảo sát:

 Khảo sát cửa hàng đồ ăn nhanh (Câu trả lời)

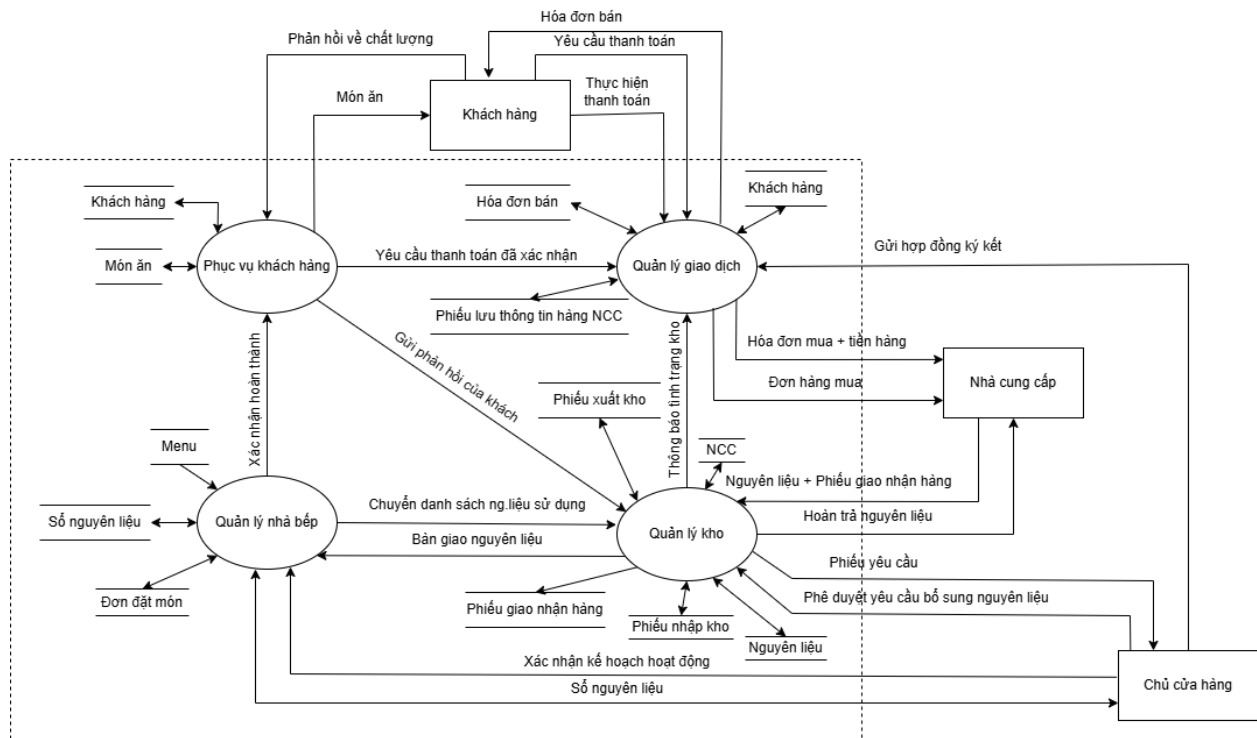
2. Hồ sơ phân tích



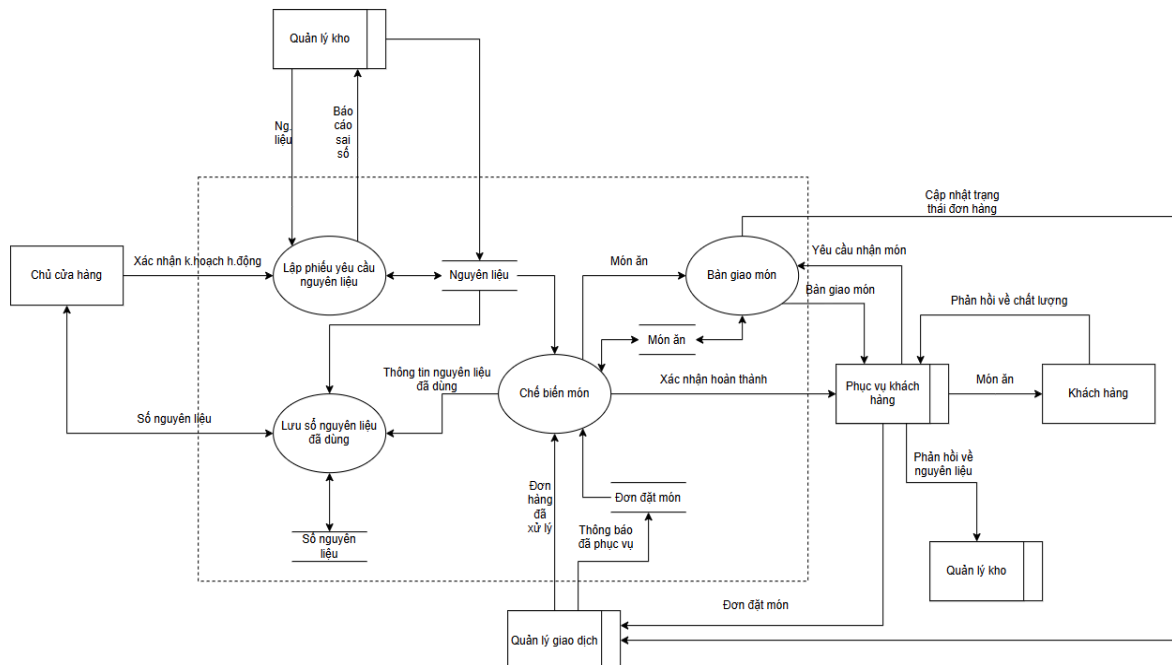
BFD hệ thống



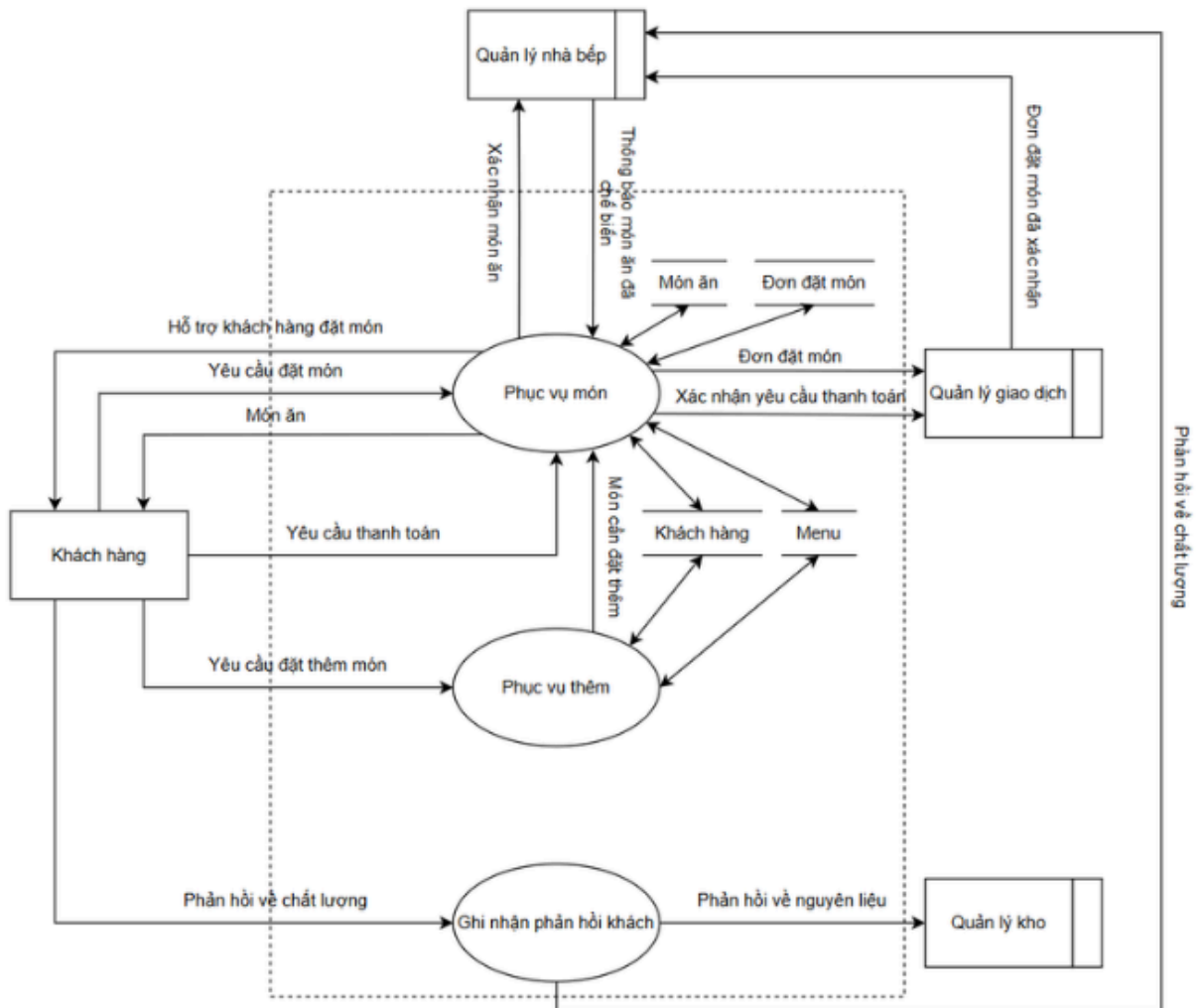
DFD mức 0



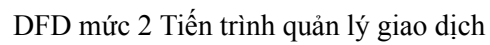
DFD mức 1



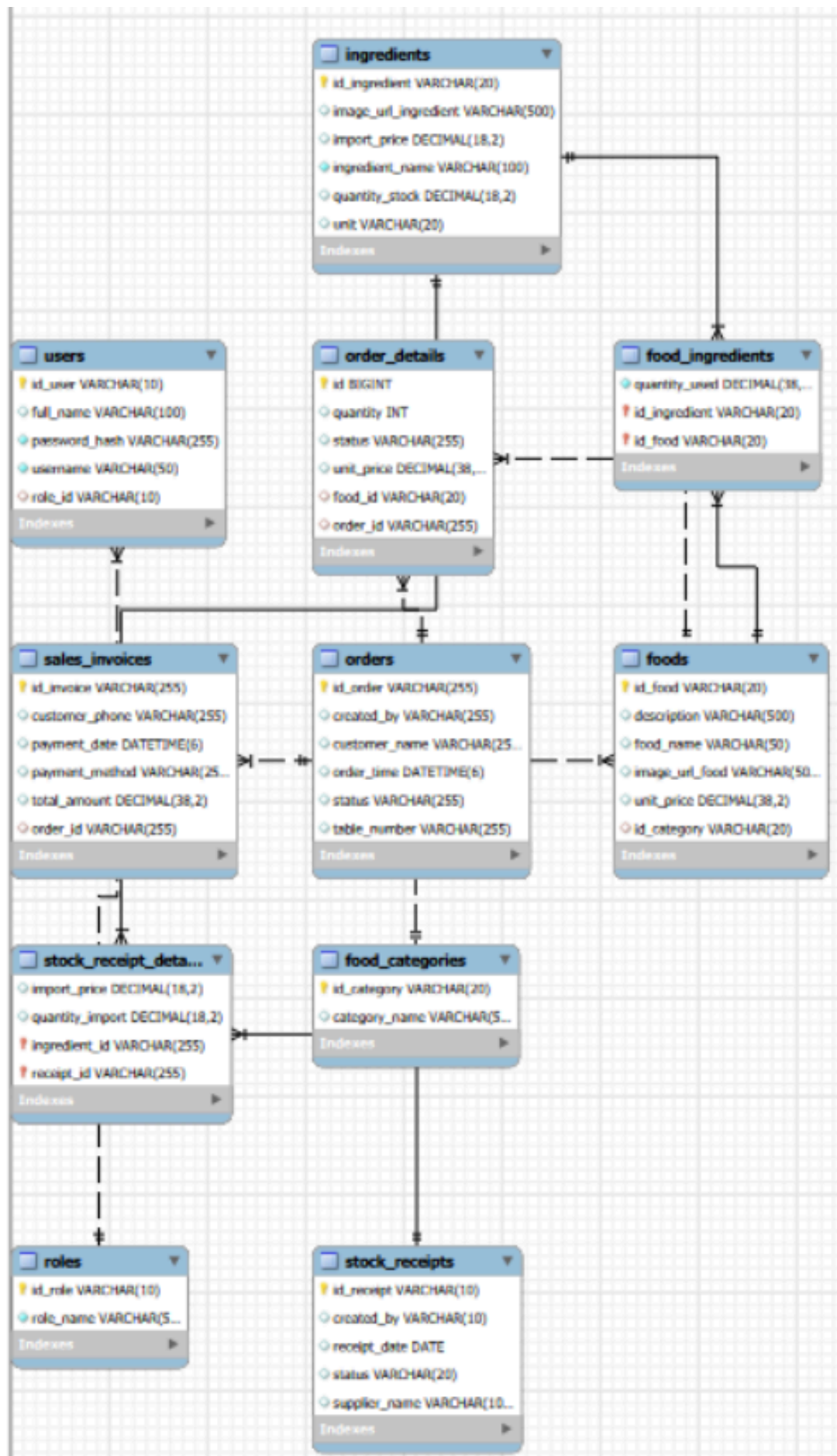
DFD mức 2 Tiến trình quản lý bếp



DFD mức 2 Tiến trình phục vụ khách hàng



Sơ đồ ERD



4. Mã nguồn

FRONTEND

```
|— index.html
|— package.json
|— package-lock.json
|— vite.config.js
|— eslint.config.js
|— README.md
└─ src
    |— components
    |— pages
    |— services
    |— assets
    |— utils
    └─ main.jsx / main.js
```

```
BACKEND
├── pom.xml
├── src
│   ├── main
│   │   ├── java/com/fastfood
│   │   │   ├── FastfoodBackendApplication.java
│   │   │   ├── config
│   │   │   ├── controller
│   │   │   ├── dto
│   │   │   │   ├── request
│   │   │   │   └── response
│   │   │   ├── entity
│   │   │   │   ├── catalog
│   │   │   │   ├── system
│   │   │   │   └── transaction
│   │   │   ├── exception
│   │   │   ├── mapper
│   │   │   ├── repository
│   │   │   ├── service
│   │   │   │   └── impl
│   │   └── resources
│   │       ├── application.properties
│   │       └── db/backfill
│   └── test
│       └── java/com/fastfood
```

5. Hồ sơ kiểm thử

Test case:

Mã TC	Chức năng kiểm thử	Mục tiêu	Điều kiện tiên quyết	Dữ liệu kiểm thử	Độ quan trọng	Kết quả mong đợi	Kết quả thực tế	Đánh giá
TC01	Đăng nhập Admin	Kiểm tra đăng nhập và giao diện Admin	Có tài khoản Admin	ADMIN/ đúng pass	Cao	Đăng nhập thành công, hiển thị giao diện Admin	Hệ thống chuyển hướng đúng trang Admin, hiển thị đầy đủ menu quản trị.	PASS
TC02	Đăng nhập Thu ngân	Kiểm tra giao diện đúng vai trò Thu ngân	Có tài khoản Thu ngân	ThuNgan / đúng pass	Cao	Hiển thị đúng màn hình thanh toán/đơn hàng	Chuyển hướng về trang Thu Ngân , hiển thị danh sách bàn và menu gọi món.	PASS
TC03	Đăng nhập Bếp	Kiểm tra giao diện đúng vai trò Bếp	Có tài khoản Bếp	Bep/ đúng pass	Cao	Hiển thị danh sách đơn cho bếp	Chuyển hướng về Bếp , danh sách đơn hàng được load realtime.	PASS

TC04	Đăng nhập sai	Kiểm tra xử lý sai mật khẩu	Có tài khoản hợp lệ	ADMIN/sai pass	Cao	Hiện thị thông báo lỗi	Popup thông báo "Sai tài khoản hoặc mật khẩu" xuất hiện đúng lúc.	PASS
TC05	Xem menu	Kiểm tra hiển thị món ăn	Có dữ liệu món	Danh sách foods	Trung bình	Hiện thị đúng tên, giá, danh mục	Hình ảnh và thông tin món ăn load mượt từ Database.	PASS
TC06	Tạo đơn hàng	Kiểm tra tạo đơn	Có dữ liệu món	Burger x2	Cao	Tạo đơn thành công, lưu DB	Record mới được tạo trong bảng orders , UI cập nhật trạng thái "Đang chờ".	PASS
TC07	Thêm món	Kiểm tra thêm món vào đơn	Đã có đơn	Khoai tây x1	Trung bình	Đơn cập nhật đúng, tổng tiền	Đơn hàng cập nhật số lượng món mới, giá trị đơn hàng nhảy số	PASS

						thay đổi	ngay lập tức.	
TC08	Gửi đơn xuống bếp	Kiểm tra luồng đơn	Đã tạo đơn	Đơn bàn 5	Cao	Hiện thị đơn ở bếp	WebSock et gửi signal thành công, màn hình Bếp hiện đơn mới sau 1s.	PASS
TC09	Cập nhật trạng thái món	Kiểm tra xử lý trạng thái	Có đơn ở bếp	Đơn bàn 5	Cao	Trạng thái cập nhật thành công	Trạng thái chuyển từ Pending sang Processin g khi Bếp click xác nhận.	PASS
TC10	Lập hóa đơn	Kiểm tra tạo hóa đơn	Có đơn hàng	Bàn 5	Cao	Tạo hóa đơn thành công	Bản ghi trong bảng Invoices được tạo, ID hóa đơn khớp với ID đơn hàng.	PASS

TC11	Tính tổng tiền	Kiểm tra tính toán	Đơn có nhiều món	2 Burger + 1 Pepsi	Cao	Tổng tiền chính xác	Logic Backend tính toán đúng (Số lượng x Đơn giá), không sai lệch dù đơn dài.	PASS
TC12	Thanh toán	Kiểm tra hoàn tất thanh toán	Có hóa đơn	Tiền mặt	Cao	Thanh toán thành công, lưu DB	Thanh toán xong sẽ gửi thông tin đơn xuống bếp, sau khi bếp phục vụ và khách ăn xong sẽ được nhân viên thu ngân xác thực và giải phóng bàn.	PASS
TC13	Nhập kho	Kiểm tra tạo phiếu nhập	Có dữ liệu NCC	Thịt bò 10kg	Trung bình	Tạo phiếu nhập thành công	Thông tin nhà cung cấp và số lượng nguyên liệu được lưu vào	PASS

							stock_receipts.	
TC14	Cập nhật tồn kho	Kiểm tra số lượng tồn	Đã nhập kho	+10kg	Trung bình	Tồn kho tăng đúng	Số lượng trong bảng ingredients tăng đúng bằng lượng nhập thực tế.	PASS
TC15	Dashbo ard	Kiểm tra số liệu	Có dữ liệu đơn	Doanh thu ngày	Cao	Số liệu khớp DB	Biểu đồ doanh thu hiển thị đúng dữ liệu tổng hợp từ các hóa đơn đã thanh toán.	PASS

Kết quả kiểm thử:

- **Tính ổn định:** Hệ thống vận hành trơn tru, không xảy ra hiện tượng treo hoặc sập Server khi thực hiện nhiều giao dịch cùng lúc.
- **Tính chính xác:** Toàn bộ dữ liệu về tài chính và tồn kho được xử lý chính xác theo đúng đặc tả yêu cầu nghiệp vụ.
- **Trải nghiệm người dùng:** Giao diện trên Tablet mượt mà, các thông báo thời gian thực giúp giảm thiểu sai sót và tăng tốc độ phục vụ tại cửa hàng.